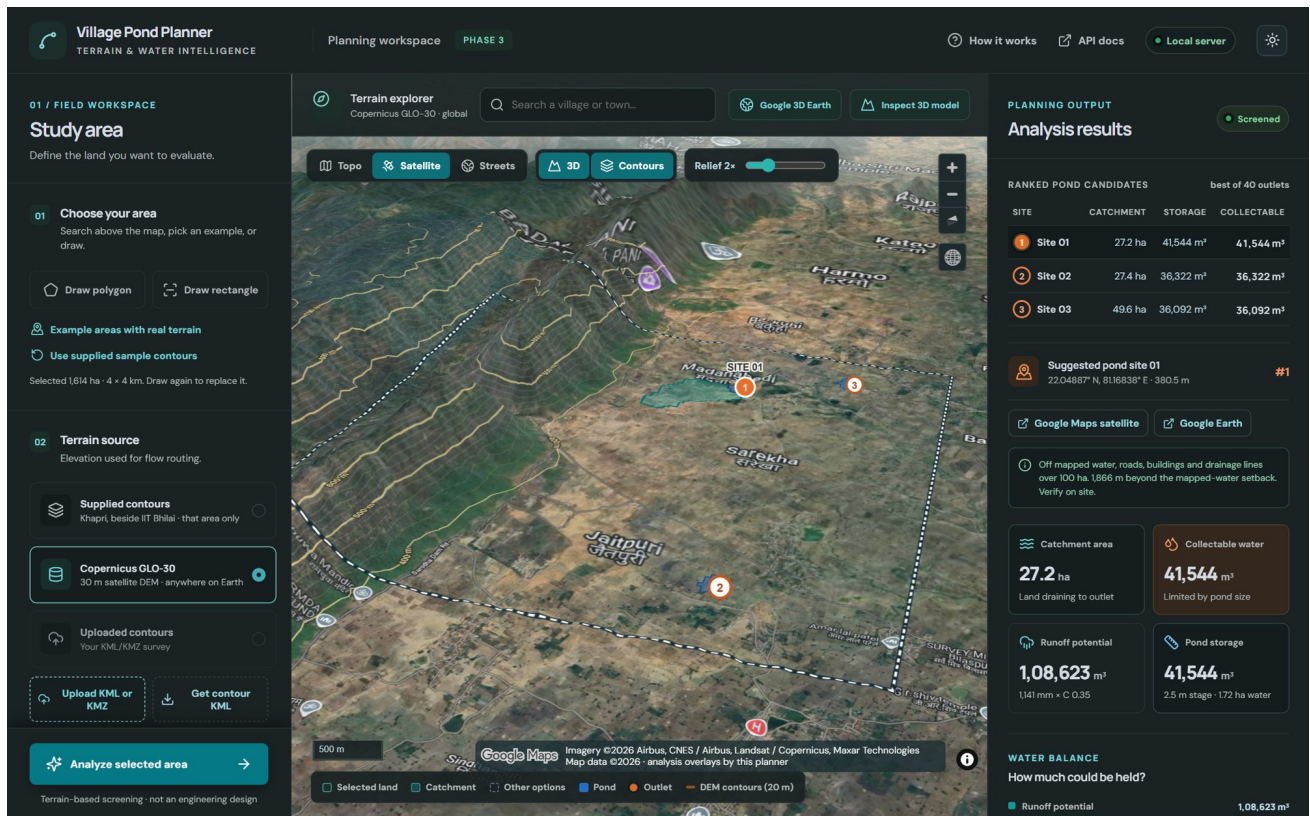




The planner in its dark theme: three ranked pond sites on the Maikal ridge front (Kabirdham), shown in tilted 3D over Google satellite imagery. The Google logo and imagery attribution stay visible in every Google view.

AI-based Village Pond Planning System

An interactive web planner that finds pond sites, their catchments and the water they could hold, for any area on Earth

DEPLOYMENT

Live on the IIT network
verified 28 Sep 2026

VALIDATION

36 automated tests
plus 5 browser suites

COVERAGE

11 verified areas
India and Ethiopia, 60°S–60°N

PLANNER

<http://10.1.75.53:3233/> (IIT Bhilai network or VPN)

API DOCS

<http://10.1.75.53:3233/docs>

PHASE 2 ROUTE

<http://10.1.75.53:3233/analyzeContour> (multipart field contour_map)

SOURCE CODE

<https://github.com/Galabavamsi/ai-village-pond-planning>

AI USE

Disclosed in Section 13

Contents

1	Executive summary	3
2	Requirements and traceability	4
3	System architecture	4
4	Features and user workflow	6
4.1	Choosing land anywhere	6
4.2	Terrain sources	6
4.3	Rainfall, land cover and pond settings	7
4.4	Results, comparison and benchmarks	9
4.5	Maps, 3D terrain and globe view	9
4.6	Google Maps Platform layers	9
4.7	3D model of the analysis grid	11
4.8	Exports	11
4.9	Dark and light themes, icons and accessibility	12
5	Algorithms	15
5.1	Coordinates and grids	15
5.2	Depression filling, D8 routing and accumulation	15
5.3	Screening	15
5.4	Candidate outlets and the impoundment model	16
5.5	Rainfall, water volume and ranking	16
5.6	Contours and 3D display	16
5.7	Pseudocode	17
5.8	Parameters	17
6	How the water volume is calculated: a worked example	18
6.1	The same calculation on a real site: Kanker west	22
6.2	Reading the result on the map	22
7	Data sources, provenance and licences	24
8	Verified example areas	25
9	Quality review: defects found and fixed	27
10	Validation	29
10.1	Automated tests	29
10.2	Live verification	30
11	Deployment on the IIT system	30
11.1	Constraints found and how they were handled	31
12	Limitations and responsible use	31
13	Disclosure of AI use	32
14	Future work	33
	References and links	33
A	API reference	35
B	Reproduce and run	36
C	Demonstration script	36
D	Repository map	37

1 Executive summary

Phase 2 delivered an API that turns an uploaded contour map into pond-site and catchment estimates. Phase 3 turns it into a complete planning tool that an administrator can use in a browser. They select land on a map, and the planner overlays three ranked pond sites. Each site comes with its upstream catchment, the water an embankment there could hold, and the runoff that a real rainfall record would deliver.

What Phase 3 delivers

- **Any area, real data.** The planner uses the supplied contour map, any uploaded KML/KMZ contour map, or the global Copernicus GLO-30 elevation model anywhere between 60°S and 60°N. Rainfall comes from CHIRPS v3 for a single month, the June–September monsoon or a full year. OpenStreetMap supplies water bodies, roads and buildings to avoid.
- **A physically consistent pond model.** Storage counts only land that drains to the pond outlet and lies below the embankment crest. Sites are ranked by the water they could actually collect. Each site reports a stage–storage curve, an indicative embankment length and warnings. Section 6 works through the whole calculation by hand on a small grid.
- **Earth view.** Topographic, satellite and street basemaps, labelled contours, 3D terrain and a globe. With a Google Maps key the satellite layer is Google imagery, a **Google Photorealistic 3D** view shows the ponds and catchments on Google’s 3D Earth, and place search uses Google Places. A 3D model of the exact analysis grid can be draped with the same imagery. Results export to Google Earth KML, GeoJSON and contour KML.
- **Dark interface.** The planner opens in a dark theme, with a one-click light theme; icons come from the open-source Lucide set.
- **Deployed and verified.** The planner runs on the assigned IIT container. All 36 tests pass, and the browser suites, including a Google-layer check, pass against the live address from the campus network.

Before this final build, the Phase 3 prototype was reviewed line by line. The review found and fixed four defects that changed results:

- pond storage was inflated by 37–272 %;
- site ranking ignored storage;
- map screening failed for larger areas;
- the map lost the selected area when the terrain source changed.

A later check found and fixed a fifth: flat terraces created when coarse contours are interpolated. These defects are documented in Section 9.

Honest scope

All numbers are deterministic screening estimates computed from gridded elevation and historical rainfall. They are not an engineering design, a hydrological simulation or a land-eligibility decision. No language model computes any result, and Google’s imagery and 3D surface are shown for context only: no planner number is derived from them. Seepage, evaporation, siltation, soil, spillway design and land ownership are not modelled, and the planner says so in every result.

2 Requirements and traceability

Phase 3 requirement	Implementation and evidence	Status
Select land on a map	Draw a polygon or rectangle (Terra Draw), pick one of 11 example areas, or search any place (Google Places, with Photon/OSM as fallback). The area and span are shown live and validated (0.25–10,000 ha, ≤ 20 km). Figs. 2, 3a.	Done
Analyze the land	POST /api/analyze-area with the supplied, uploaded or Copernicus elevation, CHIRPS or manual rainfall, and land-cover and pond settings (Sections 5 and 6).	Done
Show pond location, catchment and water volume on the map	Numbered outlets, pond polygons and catchment polygons for all three sites (active one highlighted), plus runoff, storage and collectable volume, a stage–storage chart and benchmarks (Figs. 2, 18, 6).	Done
3D maps and earth view	MapLibre 3D terrain and globe; Google Photorealistic 3D with the planner’s overlays; a 3D model of the analysis grid with elevation colours or a satellite drape (Figs. 7–9).	Done
Works beyond the sample	Copernicus GLO-30 anywhere at 60°S–60°N. Ten areas outside the sample were verified end to end (Section 8).	Done
No hard-coded results	All inputs come from the selected polygon and data source. The example list only moves the map. A test uploads and analyzes every shipped KML.	Done
Front end on the IIT system	http://10.1.75.53:3233/ on sys1 (SSH port 2233, Application 1 port), verified from the campus network (Section 11).	Done
Phase 2 API compatibility	POST /analyzeContour (field contour_map) and /findCatchment unchanged; Phase 2 tests pass.	Done
GitHub repository	https://github.com/Galabavamsi/ai-village-pond-planning , branch master. The Google key is not in the repository.	Done
Report and demo video	This report; the video follows the walkthrough in Appendix C.	Report done

3 System architecture

The planner is one FastAPI application that serves both the API and the compiled React front end, so there is a single origin and a single process to run. Figure 1 shows the parts and the data they use.

Layer	Technology (version)
Front end	React 19, TypeScript 5.9, Vite 7, MapLibre GL JS 6.11 (globe, terrain, sky), Terra Draw 1.35, Three.js 0.186, Lucide icons, Google Maps JavaScript API (Map3DElement, loaded only when the 3D Earth view opens)
API	FastAPI 0.141, Uvicorn, Pydantic request models, OpenAPI/Swagger at /docs
Geospatial	NumPy 2.5, SciPy 1.18 (interpolation, morphology), Shapely 2.1, rasterio 1.5 with GDAL 3.12, contourpy 1.4, pystac-client, planetary-computer
Testing	pytest (36 tests), Playwright with installed Chrome (five browser suites, desktop and mobile)
Hosting	Ubuntu 24.04 container on the IIT system, Python 3.12 virtual environment, tmux supervision

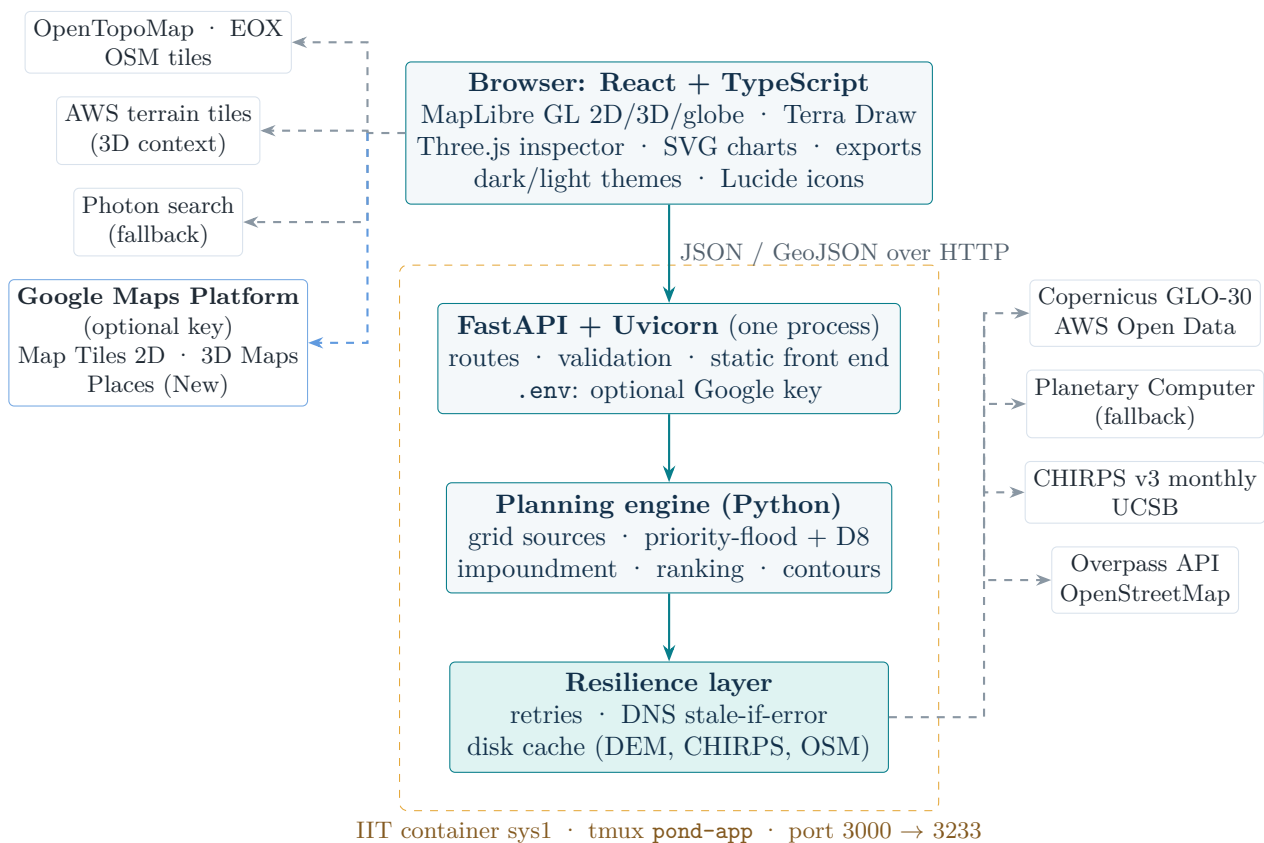


Figure 1: Architecture. Solid arrows are the planner’s own calls. Dashed arrows are remote data sources: the browser loads map tiles, terrain tiles, search and the optional Google layers itself, and the server reads elevation, rainfall and OSM features. Each server-side source is retried and cached. The server only hands the Google browser key to the page (from a git-ignored `.env`); it never calls Google itself.

4 Features and user workflow

The screenshots in this section and in Section 6 were captured on 28 September 2026 from a local run of the same build that is deployed, so the header badge reads “Local server”. Live captures over the campus Wi-Fi came out unstyled or blank, because the link drops about half of new connections (Section 11). The same browser suites pass against the live address (Section 10).

4.1 Choosing land anywhere

The workspace has three columns: the study-area controls, the map, and the results (Figure 2). A planner can:

- **draw** a polygon or rectangle on the map;
- **search** a village, town or landmark, which selects a 2 km square around it (Figure 3b);
- **pick an example area** with real terrain (Figure 3a).

The panel shows the selected area and its width and height in kilometres live, and rejects impossible sizes before any request is sent. Drawing outside the supplied or uploaded contours switches the terrain source to Copernicus GLO-30 automatically and explains why in a notice.

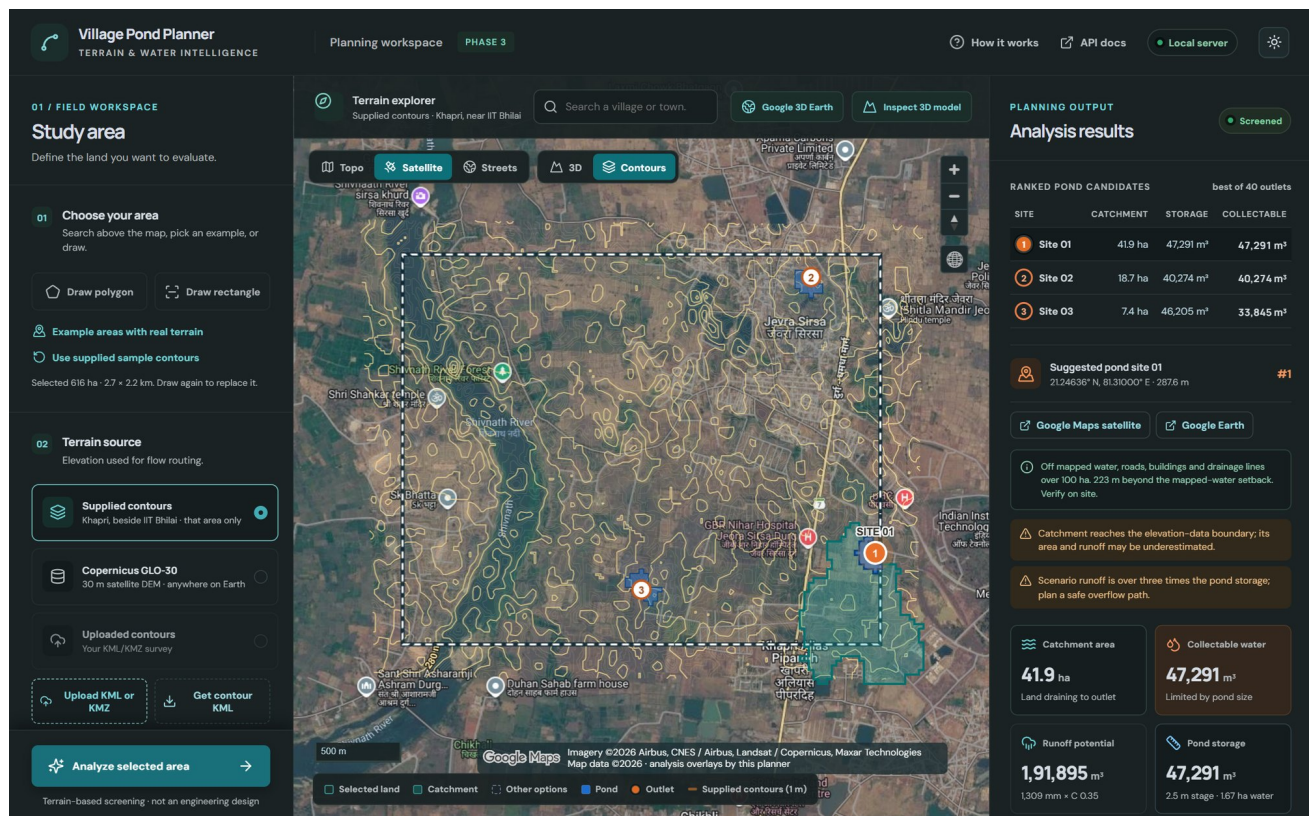


Figure 2: Supplied contour map (Khapri, beside IIT Bhilai) analyzed with the June–September 2025 CHIRPS rainfall, in the default dark theme over Google satellite imagery. Three numbered outlets, their ponds (blue) and catchments (teal for the active site, dashed for the alternatives) sit over the 1 m contours. The results column ranks the sites by collectable water and lists warnings. Figure 18 labels each element of the map.

4.2 Terrain sources

- **Supplied contours:** the course KML (2,710 lines, 267–298 m). Section 7 explains its provenance.
- **Uploaded contours:** any KML or KMZ up to 20 MB. Each line’s elevation is read from its place-mark name, an `ExtendedData` field (`elevation`, `elev`, `z`, `height` or `level`), or a constant coordinate altitude. The uploaded linework is drawn on the map (Figure 4).

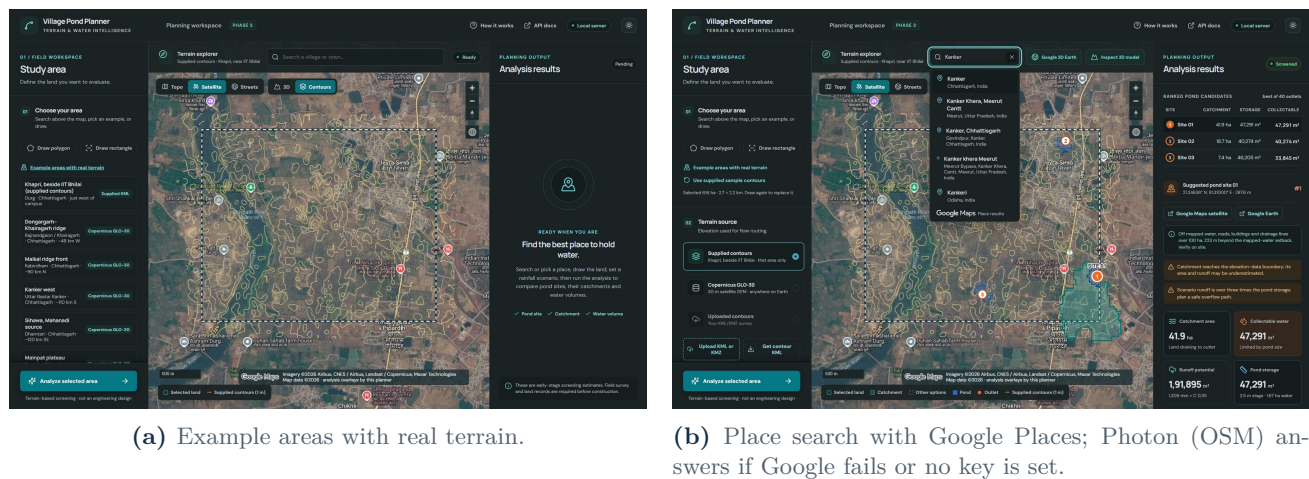


Figure 3: Two quick ways to reach an area other than the sample.

- **Copernicus GLO-30:** a 30m global surface model, read for the selected area plus a buffer, so catchments can extend beyond the drawn land.
- **Get contour KML:** traces real contours for *any* selected area. The file opens in Google Earth and can be uploaded back as a contour survey.

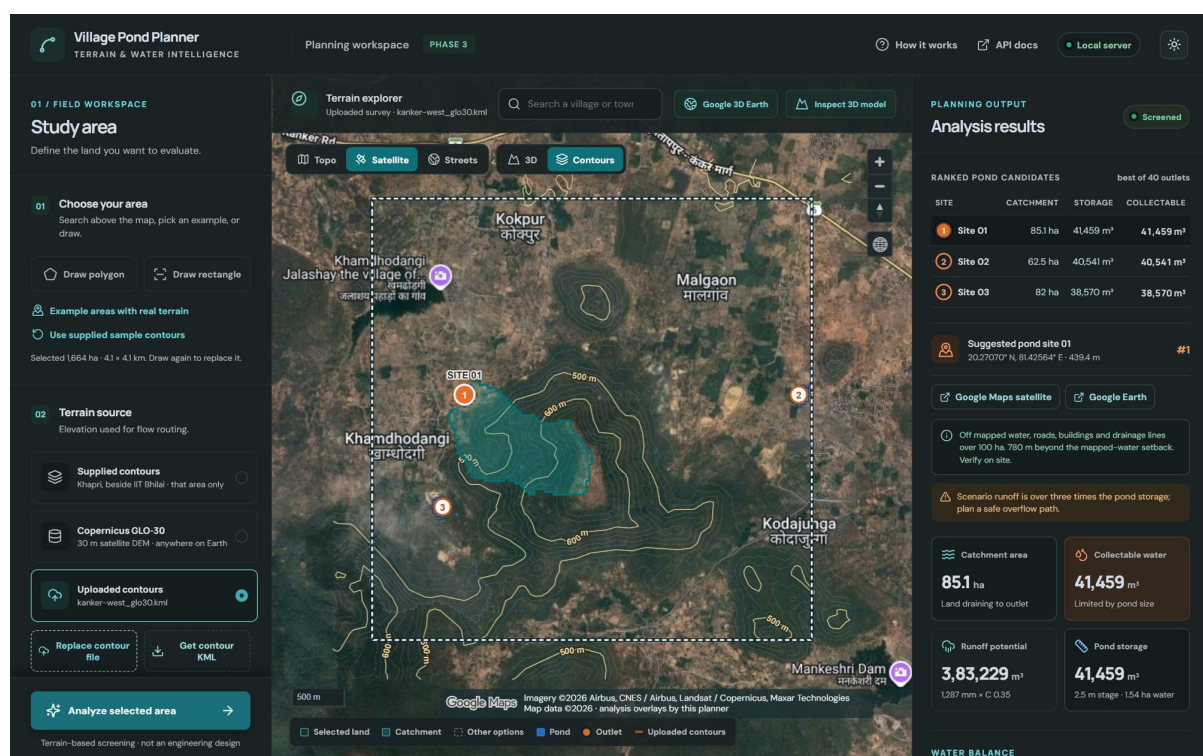


Figure 4: A real GLO-30-derived contour KML for Kanker west, uploaded and analyzed. The linework is drawn from the file, and the three outlets drain different valleys.

4.3 Rainfall, land cover and pond settings

CHIRPS v3 provides a historical monthly rainfall total on a 0.05° grid. The planner can use one month, the June–September season (the Indian south-west monsoon, when most runoff occurs) or a calendar year. Only completed months are offered. Alternatively the user enters a rainfall depth. The runoff coefficient has land-cover presets within the ranges of the CGWB *Manual on Artificial Recharge* [10] (Table 1). The water stage (embankment height, 0.5–6 m) sets the pond size. An advanced setting caps the catchment treated as a “small pond” at 100 ha by default.

01 / FIELD WORKSPACE

Study area

Define the land you want to evaluate.

01 Choose your area

Search above the map, pick an example, or draw.

Draw polygon

Draw rectangle

Example areas with real terrain

Use supplied sample contours

Selected 1,604 ha · 4 × 4 km. Draw again to replace it.

02 Terrain source

Elevation used for flow routing.

Supplied contours

Khapri, beside IIT Bhilai · that area only

Copernicus GLO-30

30 m satellite DEM · anywhere on Earth

Uploaded contours

Your KML/KMZ survey

Upload KML or KMZ

Get contour KML

"Get contour KML" traces real contours for any selected area; the file opens in Google Earth and can be uploaded here.

03 Rainfall scenario

Water delivered to the pond.

CHIRPS v3 history

Enter rainfall

Jun-Sep

Full year

One month

Monsoon season of

2025

Sum of June–September CHIRPS totals; most Indian runoff falls then.

04 Land cover & pond

Runoff fraction and embankment height.

Forest 0.15

Paddy 0.2

Scrub 0.25

Hilly 0.35

Rocky 0.5

Built-up 0.75

Runoff coefficient

0.35

Fraction of rainfall that runs off the catchment.

Pond water stage

2.5 m

Embankment height above the outlet ground; sets pond size.

Advanced

Analyze selected area →

Terrain-based screening · not an engineering design

Figure 5: The full study-area panel, split into two columns: 01 area selection with the live size check; 02 terrain source and the contour-KML export; 03 rainfall period and year; 04 land-cover presets, pond stage and the advanced catchment cap.

Preset	C	Basis (CGWB manual)
Forest	0.15	tree cover, volumetric range 0.05–0.2
Paddy	0.20	flat bunded cropland; Barlow 10–20 %
Scrub	0.25	grass, scrub and fallow land
Hilly	0.35	hills and plains with little cultivation; Barlow 35 %
Rocky	0.50	bare or rocky slopes, about 0.4–0.7
Built-up	0.75	roofs, roads, paved yards

Table 1: Runoff-coefficient presets. They are seasonal fractions for screening, not peak-flow design values.

4.4 Results, comparison and benchmarks

The results panel (Figure 6) supports the decision the administrator actually has to make:

- a **comparison table** of the three alternatives;
- **metrics** for the active site: catchment area, collectable water (and whether runoff or pond size limits it), runoff potential and pond storage;
- a **water balance** with bars on one shared scale;
- **pond facts**: crest level, mean depth, indicative bund length and storage per metre of bund;
- **benchmarks** against Mission Amrit Sarovar and MGNREGA farm-pond norms [8,9];
- a **stage–storage chart** with a crosshair tooltip and a table view;
- **monthly rainfall bars**;
- **links** that open the outlet in Google Maps (satellite) or Google Earth; they need no key;
- data **provenance**, and **exports**.

Each site also carries plain-language warnings: its catchment reaches the grid edge, it drains more than the small-pond cap, or runoff is more than three times, or less than half, its storage.

4.5 Maps, 3D terrain and globe view

The map toolbar switches between three basemaps: OpenTopoMap topography, satellite imagery and OpenStreetMap streets. The satellite layer is Google imagery with road and place labels when a Google key is configured, and EOX Sentinel-2 cloudless 2025 otherwise. The toolbar also toggles contours and **3D**, which tilts the map over global terrain tiles with adjustable relief (cover image). The globe button zooms out to the whole Earth (Figure 7). Contours are labelled every fifth line; over imagery the minor lines are fainter so the index lines read. Before analysis they come from the survey; after a Copernicus analysis they are traced from the analysis grid. The map is created once and updated in place, so switching source or basemap never loses the selection.

4.6 Google Maps Platform layers

The Google layers are optional. The server reads a browser key from a git-ignored `.env` file and passes it to the page in `/api/config`; without it every Google feature falls back to the open sources above.

- **Satellite basemap** (Map Tiles API, 2D tiles). The planner opens a tile session, draws the Google logo on the map, and asks the viewport endpoint for the copyright text of the imagery actually in view after every move, as the tile policies require [6].
- **Google 3D Earth** (Maps JavaScript API, Map3DElement). Google’s Photorealistic 3D surface with the planner’s selected land, catchments, ponds and numbered outlets drawn on top as 3D polygons,

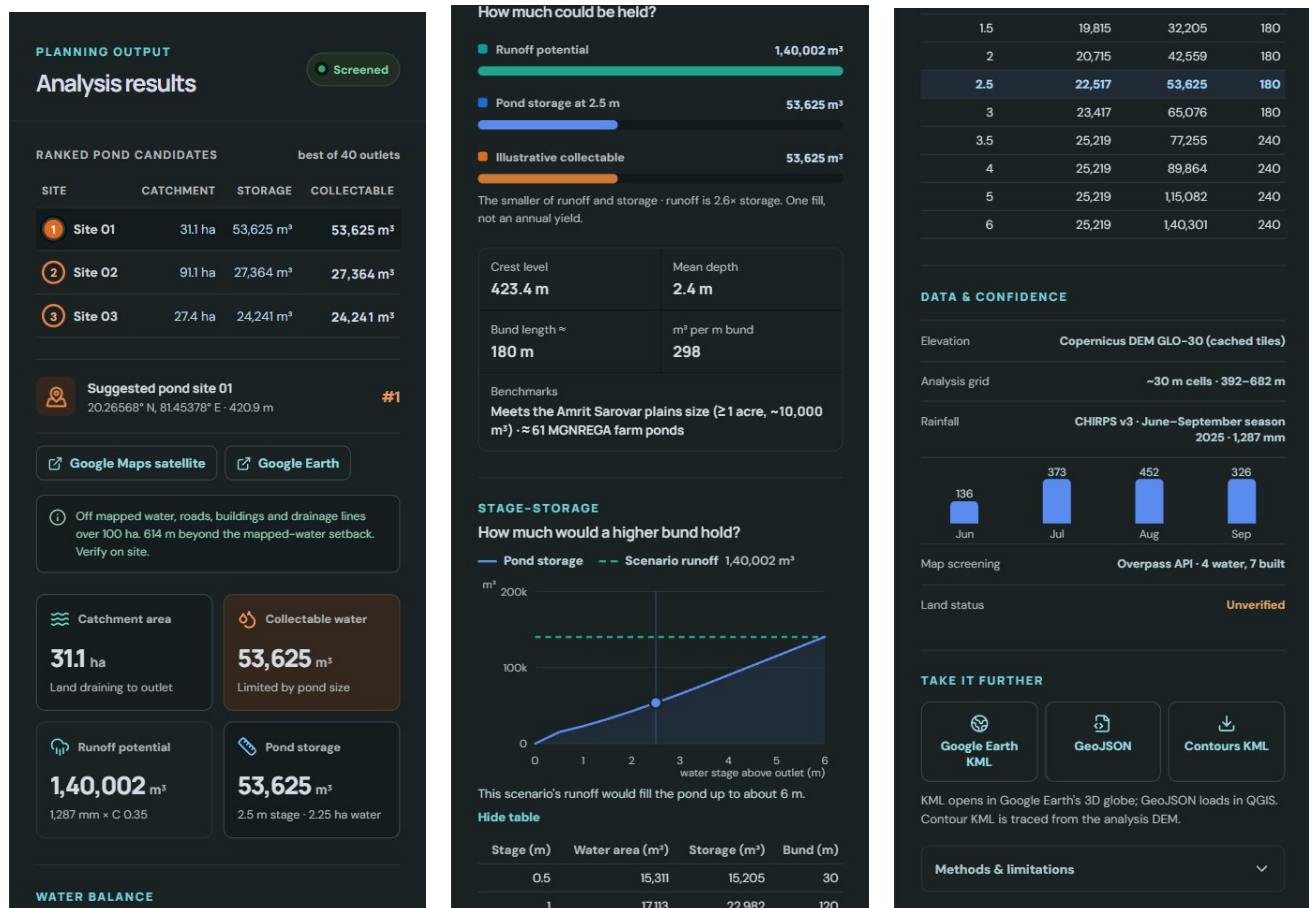


Figure 6: The complete results panel for Kanker west, split into three columns. From left: ranked comparison, links and metrics; water balance, pond facts, benchmarks and the stage-storage chart with its table view; data provenance with the monthly CHIRPS bars, and exports. Section 6.1 reproduces these numbers by hand.

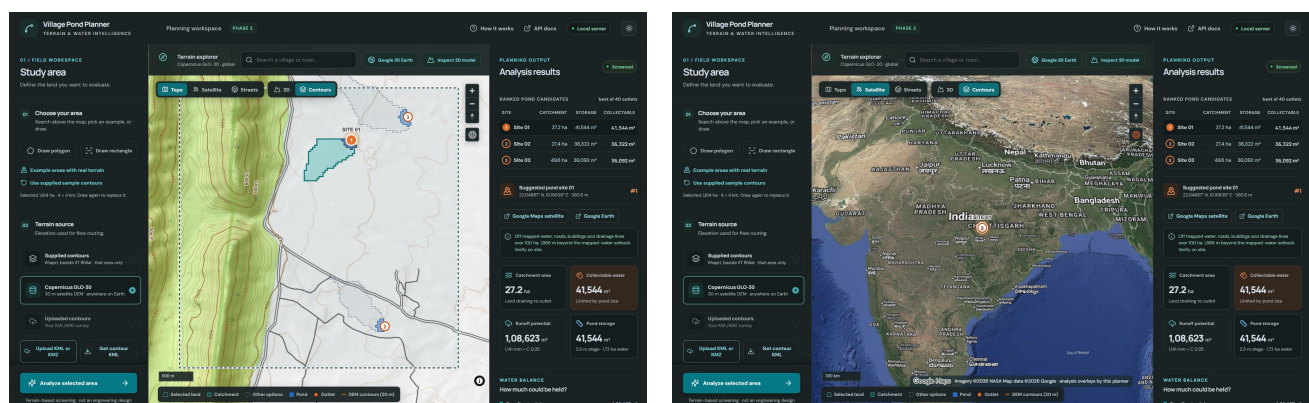


Figure 7: The same analysis on a 2D topographic map and on the globe. The cover shows it in tilted 3D over satellite imagery.

polylines and markers (Figure 8). Choosing a site flies the camera to it. The library is loaded only when this view is opened.

- **Place search** (Places API (New) autocomplete and details). Keystrokes and the final lookup share one session token, as billed sessions require. Photon answers if Google fails.
- **Satellite drape** for the 3D model (Figure 9), using the same tile session.

Google content is display-only. Nothing is cached, stored or analysed, and every number still comes from the planner's own elevation grid. Each Google view keeps the logo and attribution, and so does every screenshot in this report. If a Google layer fails (quota, key or network), an error boundary keeps the rest of the planner running.

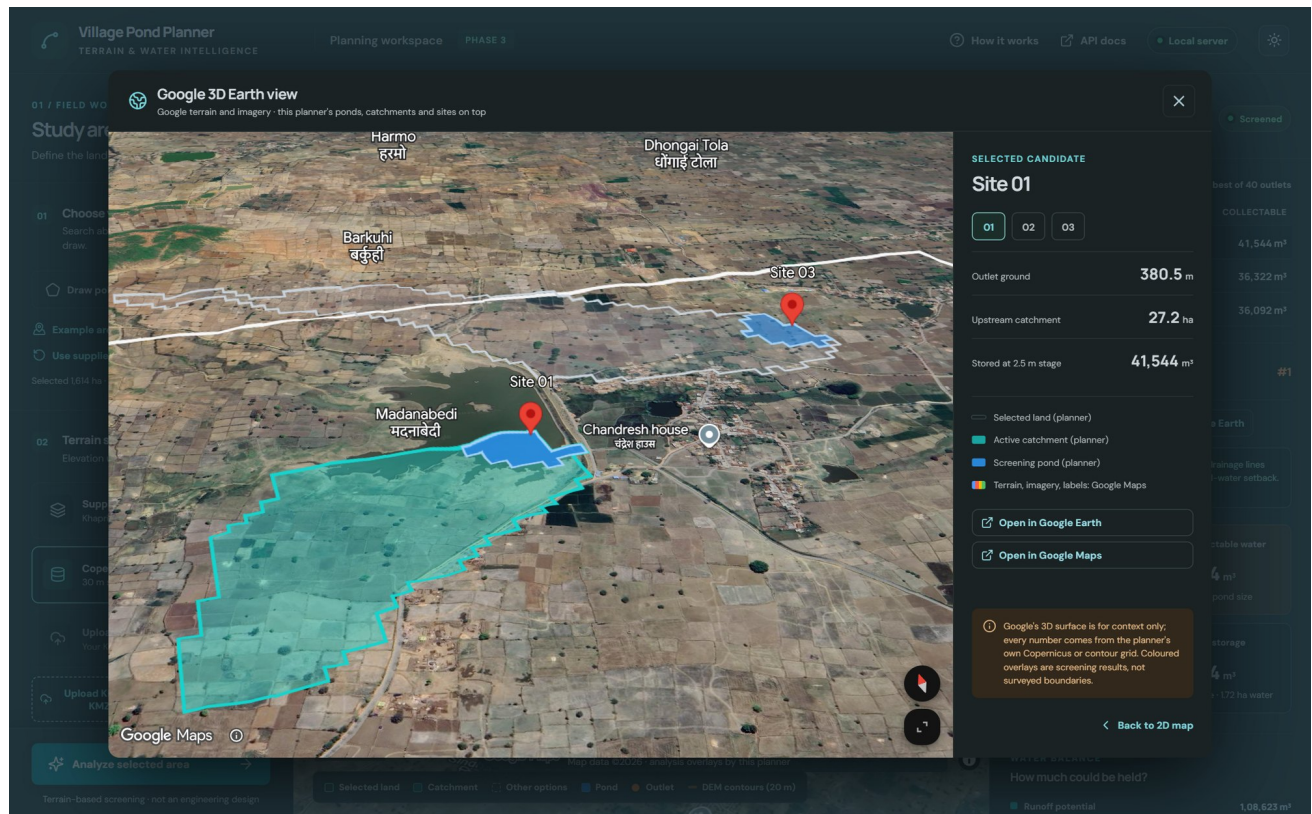


Figure 8: Google 3D Earth view of the Maikal ridge front. The teal catchment, blue pond and red outlet markers are the planner's screening results, drawn on Google's Photorealistic 3D surface. The side panel repeats the site's numbers and links to Google Earth and Google Maps.

4.7 3D model of the analysis grid

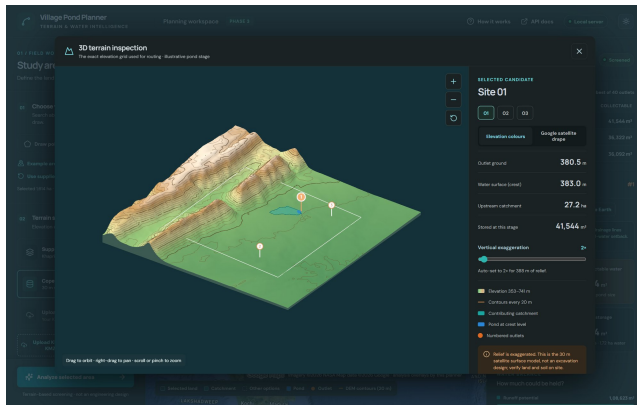
Inspect 3D model opens a Three.js rendering of the *same* elevation grid used for routing (downsampled to at most 120×120), not a separate terrain service. It shows:

- a hypsometric colour ramp with contours draped on the surface;
- the study-area outline, the tinted catchment, the pond surface at crest level, and numbered outlets for all three sites.

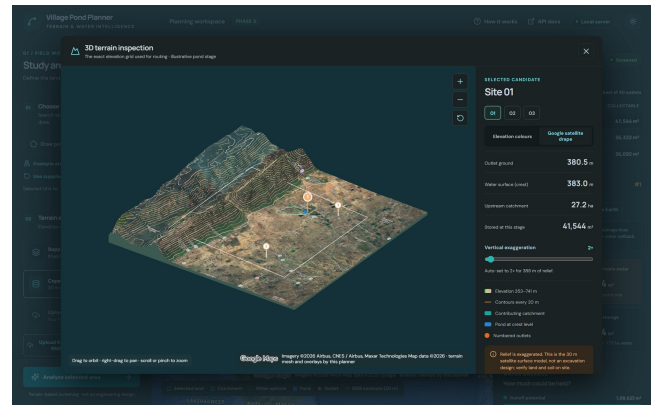
The vertical exaggeration is set automatically so that relief spans about 12% of the model width (Equation 10); a slider adjusts it. **Satellite drape** textures the mesh with Google imagery, or Sentinel-2 without a key (Figure 9). The exaggeration changes only the drawing, never the numbers.

4.8 Exports

- **Google Earth KML:** the study area, three catchments, ponds and outlets, styled and clamped to terrain, with every metric in each placemark's description. It opens directly in Google Earth's 3D



(a) Elevation colours with contours and catchment.



(b) Google satellite drape on the same mesh, with its attribution.

Figure 9: 3D inspector for the Maikal ridge front.

globe.

- **GeoJSON:** the same features with numeric properties, for QGIS or other GIS tools.
- **Contour KML:** real contours for the selected area at a relief-based interval, with provenance and the Copernicus attribution in the file.

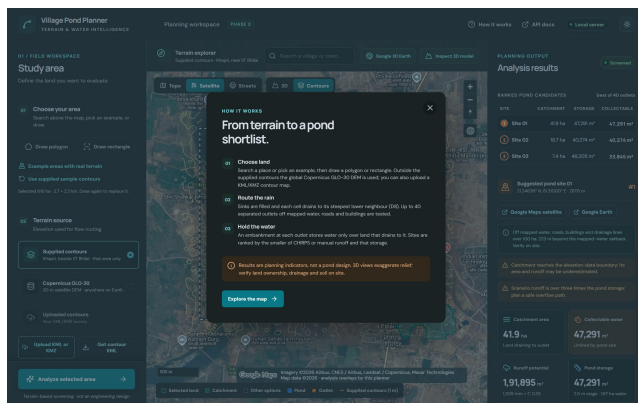
4.9 Dark and light themes, icons and accessibility

The planner opens in a dark theme, which suits imagery and long sessions; the sun/moon button in the header switches to the light theme (Figure 10) and the choice is remembered. The dark colours are not hand-picked one by one. `scripts/build_dark_theme.py` reads every colour in the light stylesheets and maps it by the role it plays (surface, text, border, chart mark or shadow) in the perceptual OKLCH colour space [14]:

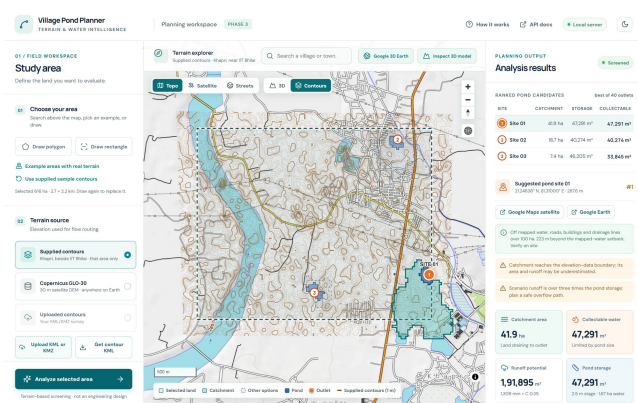
- surfaces become dark but keep the light theme's order of elevation;
- text becomes light with at least 4.5:1 contrast;
- brand fills keep their hue, and pale gridlines stay recessive.

Hand-tuned rules (map controls, the white Google logo on dark surfaces) live in `dark-extra.css`. The chart series (blue storage, teal runoff, orange collectable) were given their own dark-mode steps and checked with a palette validator on the dark surface: all three lie in the dark lightness band; the worst adjacent colour-blind separation is $\Delta E = 11.9$ and the normal-vision separation 18.8 (targets 8 and 15); each has at least 3:1 contrast. Icons are the open-source Lucide set throughout.

In the light theme, about 30 grey text colours below 4.5:1 were darkened and the minimum text size raised. Charts have legends and table views, so colour is never the only cue. The help dialog and the 3D views close with Escape. On phones the controls slide in from the side, and the map, results and a sticky **Analyze** button stack vertically (Figure 11).



(a) Dark theme: the “How it works” dialog.



(b) Light theme with the topographic basemap.

Figure 10: The two themes. The light theme is the original, contrast-checked design; the dark theme is generated from it.

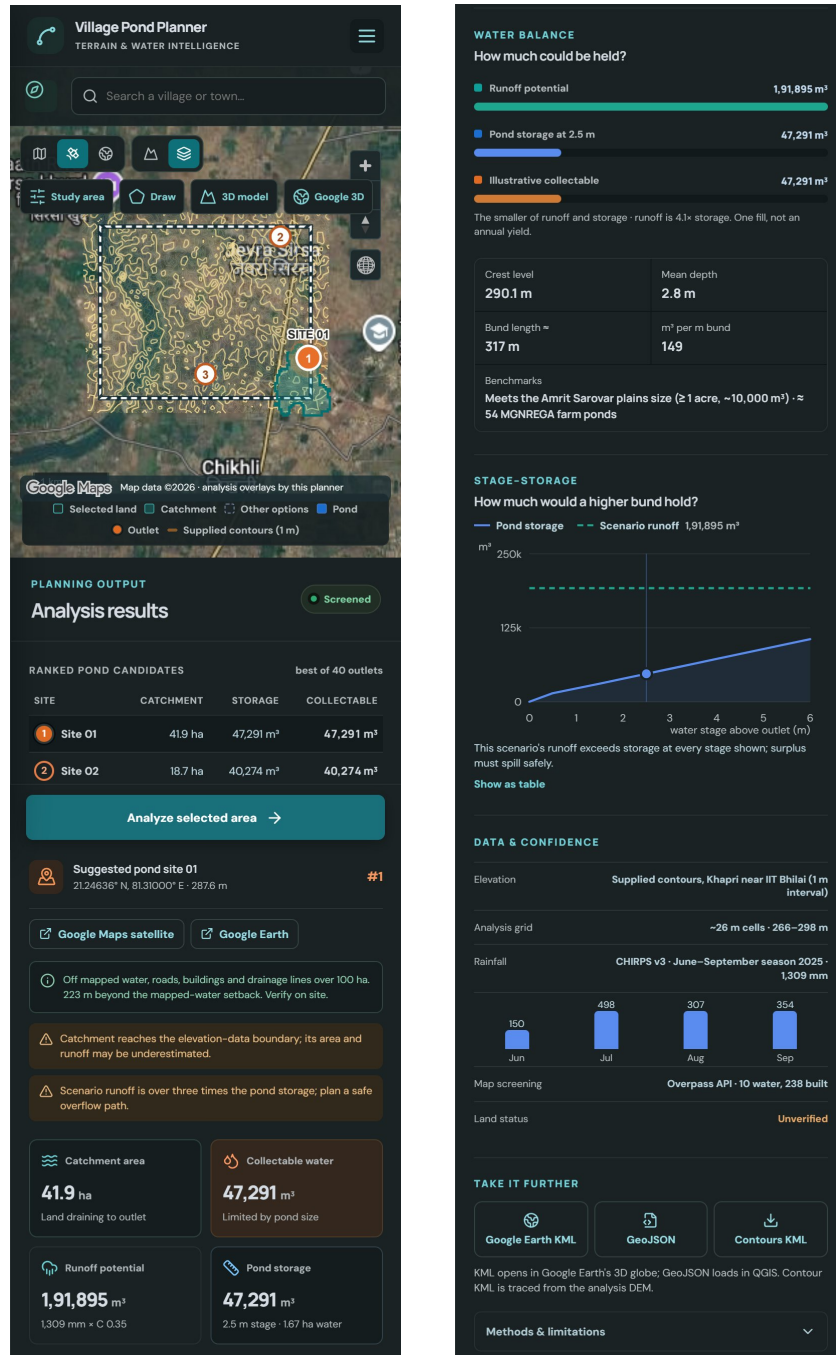


Figure 11: Mobile layout (390 px wide), shown in two halves.

5 Algorithms

The analysis is deterministic. Given the same polygon, elevation source and settings it returns the same result, so results can be checked and reproduced. Algorithm version: `priority-flood-d8-impoundment-v4`. This section states the method; Section 6 applies every step to a small grid with the numbers written out.

5.1 Coordinates and grids

The polygon is validated: a simple polygon of at most 1,000 vertices, 0.25–10,000 ha, and no wider than 20 km. Longitude λ and latitude ϕ are projected to local metres around an origin (λ_0, ϕ_0) with $R = 6,371,000$ m:

$$x = R \cos \phi_0 (\lambda - \lambda_0), \quad y = R (\phi - \phi_0), \quad A_{\text{cell}} = |\Delta x \Delta y|. \quad (1)$$

Contour sources. Contour vertices are interpolated to a regular grid of up to 220 cells along the longer side (the sample uses 125). A linear (triangulated) surface is flat wherever all three vertices of a triangle lie on one contour, which happens on hilltops and valley floors. With 20 m contours, 22.5 % of cells were flat, and a “pond” there became a flat patch whose storage was simply footprint $\times$ stage. Phase 3 therefore uses a C^1 Clough–Tocher surface z_{CT} [13], bounded by the contour interval Δ :

$$\tilde{z} = \text{clip}(z_{\text{CT}}, z_{\text{lin}} - \Delta, z_{\text{lin}} + \Delta). \quad (2)$$

Between two drawn contours the ground cannot cross another contour level, so this bound keeps the smooth surface consistent with the survey. It removes the overshoot that unbounded cubic interpolation produces (318 m on a survey whose top contour is 298 m). Outside the linework’s convex hull the nearest value is used. The Phase 2 route keeps linear interpolation, so its published results stay reproducible. Figure 12 illustrates the difference.

Copernicus source. The selected area’s bounding box is buffered by 30 % of its span, and by at least 0.009° (≈ 1 km). The 1° GLO-30 Cloud-Optimized GeoTIFFs that cover it are read from AWS Open Data and resampled bilinearly to at most 280×280 cells (≈ 30 m). Any gap is filled from Microsoft Planetary Computer’s copy. A box that crosses tile seams was tested and has no gaps.

5.2 Depression filling, D8 routing and accumulation

Depressions are filled with a priority flood from the grid edge [11]. Cells are visited in order of spill elevation, and each unvisited neighbour n of the current cell c is raised to at least $z'_c + \varepsilon$ ($\varepsilon = 10^{-5}$ m). Flats therefore drain without cycles:

$$z'_n = \max(z_n, z'_c + \varepsilon). \quad (3)$$

Each cell i drains to the neighbour $n \in N_8(i)$ with the steepest downhill grade (the D8 rule [12]), where $d(i, n)$ is the orthogonal or diagonal distance:

$$f(i) = \arg \max_{n \in N_8(i), z'_n < z'_i} \frac{z'_i - z'_n}{d(i, n)}. \quad (4)$$

Contributing area accumulates along this graph in topological order:

$$A(i) = A_{\text{cell}} + \sum_{k: f(k)=i} A(k), \quad C(o) = \{i : \text{the flow path from } i \text{ reaches } o\}. \quad (5)$$

5.3 Screening

OpenStreetMap features for the area (plus a margin of about 200 m) are read from the Overpass API and turned into exclusion masks (Table 2). Each exclusion is inflated by half a cell diagonal, so that no retained cell footprint crosses a mapped boundary. A partial or timed-out Overpass answer is treated as a failure, because it could silently miss water; the result is then labelled *unverified*. Drainage lines with more than A_{max} of upstream area (default 100 ha), and their neighbours, are not used as small-pond outlets: a pond there would need an engineered spillway.

Mapped feature (OSM tags)	Exclusion buffer
Water bodies, wetlands, reservoirs, basins, water=*	40 m setback
Rivers and canals / streams, ditches and drains	45 m / 25 m
Buildings and built-up land use (residential, commercial, industrial, retail, military, cemetery)	12 m
Motorway, trunk / primary / secondary / tertiary	35 / 30 / 25 / 18 m
Unclassified, residential, road / service, living street	12 / 8 m
Tracks, paths, footways and other minor ways / railways	4 / 30 m

Table 2: Approximate setbacks. They are screening margins, not legal setbacks. The prototype used a flat 35 m for every road, including footpaths, which removed large amounts of village land.

5.4 Candidate outlets and the impoundment model

Candidate outlets are cells that:

- lie inside the selected land, off the exclusions and at least two cells from the grid edge;
- drain between $\max(1 \text{ ha}, 5A_{\text{cell}})$ and A_{max} .

Taken in order of contributing area, up to 40 are kept, each at least $\max(200 \text{ m}, 3 \text{ cells})$ apart.

For an outlet o at ground elevation z_o and water stage h , an embankment would hold water up to the crest $h_c = z_o + h$. The flooded set P contains every cell j that is connected to o by 4-neighbour steps within P and satisfies all of:

$$j \in C(o), \quad z_j \leq h_c, \quad j \text{ allowed}, \quad \|\mathbf{x}_j - \mathbf{x}_o\| \leq r, \quad (6)$$

where $r = \max(100 \text{ m}, 4 \text{ cells})$ keeps the pond at village scale. Storage, footprint and indicative embankment length are then

$$V_S = \sum_{j \in P} (h_c - z_j) A_{\text{cell}}, \quad A_P = |P| A_{\text{cell}}, \quad L_E = \sum_{\substack{(j,k): j \in P, k \notin C(o) \\ z_k \leq h_c}} \ell_{jk}, \quad (7)$$

where ℓ_{jk} is the length of the shared cell edge. Lower ground *outside* the catchment is where water would escape, so its edge length is the bund that has to be built. Repeating the calculation for $h = 0.5, 1, \dots, 6 \text{ m}$ gives the stage-storage table. The flooding uses the raw grid z , not the filled z' : a real depression inside the pond holds water. The Phase 3 prototype flooded every connected low cell, including cells downstream of the outlet that a pond cannot hold. Section 9 shows the effect.

5.5 Rainfall, water volume and ranking

With rainfall depth P_{mm} (CHIRPS at the area centroid, or manual), catchment area A_C and runoff coefficient C :

$$V_R = \frac{P_{\text{mm}}}{1000} A_C C, \quad V_c = \min(V_R, V_S), \quad F = V_R/V_S. \quad (8)$$

V_c is the illustrative collectable volume for one fill, and F is the fill ratio. The 40 candidates are sorted by the key

$$(V_c \downarrow, V_S/L_E \downarrow, A_C \downarrow).$$

Three sites are then chosen so that their ponds never overlap and, where possible, no two share more than 60 % of their catchments; this rule was added when two top sites turned out to be on the same drainage line. The three sites are alternatives, so their runoff is not added together.

5.6 Contours and 3D display

Contours are traced from the analysis grid by marching squares (contourpy) and simplified to a quarter of a cell. The interval is the smallest “nice” value at or above one twenty-fifth of the relief:

$$\Delta_c = \min\{v \in \{0.5, 1, 2, 2.5, 5, 10, 20, 25, 50, 100, 200\} : v \geq (z_{\text{max}} - z_{\text{min}})/25\}. \quad (9)$$

Every fifth line is marked as major. For a model of width S and relief Z , the 3D exaggeration is

$$e = \text{clamp}(\text{round}(0.12 S/Z), 1, 30). \quad (10)$$

This replaces the prototype’s fixed $15\times$, which made flat plains look like spikes.

5.7 Pseudocode

```
validate polygon (area, span, vertices)
grid <- supplied/uploaded contours (bounded smooth interpolation) or Copernicus GL0-30 (AWS, PC
  fallback, cached)
in parallel: OSM exclusions (Overpass, retried, cached); CHIRPS totals for the chosen months (retried,
  cached)
z' <- priority_flood(grid.z); f <- steepest_D8(z'); A <- accumulate(f)
allowed <- inside polygon AND NOT (mapped water OR built land)
outlets <- separated cells with 1 ha <= A <= A_max, allowed, not beside a major channel (<= 40)
for each outlet o:
  C(o) <- upstream cells; P <- flood_fill(o, crest = z_o + h, within C(o), allowed, radius r) # on
  raw z
  V_S, A_P, L_E, stage table; V_R = P_mm/1000 * A_C * C; V_c = min(V_R, V_S)
rank by (V_c, V_S/L_E, A_C); pick 3 with non-overlapping ponds and distinct catchments
return sites, GeoJSON geometries, contours, 3D preview, rainfall, screening status and limitations
```

5.8 Parameters

Parameter	Default	Where it is set
Water stage h	2.5 m (0.5–8)	slider, <code>stage_m</code>
Runoff coefficient C	0.35 (0–1]	presets and slider, <code>runoff_coefficient</code>
Small-pond catchment cap $A_{\max}$	100 ha (5–5,000)	advanced slider, <code>max_catchment_ha</code>
Rainfall	CHIRPS Jun–Sep 2025	<code>rainfall_period</code> , <code>rainfall_year</code>
Pond search radius r	max(100 m, 4 cells)	fixed
Minimum catchment	max(1 ha, 5 cells)	fixed
Outlet separation / candidates	max(200 m, 3 cells) / 40	fixed
Copernicus buffer / grid	30 % (≥ 1 km) / $\leq 280^2$ cells	fixed

6 How the water volume is calculated: a worked example

This section follows one pond through the whole method, the way a textbook would. The grid is small enough to check by hand: 7×7 cells of 20 m, so each cell covers $A_{\text{cell}} = 20 \times 20 = 400 \text{ m}^2$. It is a synthetic valley that drains south, with one pit on its floor. Every number, arrow and depth below was computed by the planner's own functions; `scripts/report_figures.py` runs them and writes the figures, so the text cannot drift from the code:

- `_condition_dem`: priority flood;
- `_flow_graph`: D8 directions and accumulation;
- `UpstreamIndex`: catchments;
- `impoundment` and `stage_curve`: pond storage;
- `_grid_from_contours`: contour interpolation.

The same steps are then repeated for a real site in Section 6.1.

The whole calculation in one line

$$V_c = \min \left(\underbrace{\frac{P_{\text{mm}}}{1000} C A_C}_{\text{runoff } V_R}, \underbrace{\sum_{j \in P} (h_c - z_j) A_{\text{cell}}}_{\text{storage } V_S} \right), \quad h_c = z_o + h.$$

Everything else in the method decides *which* cells form the catchment $C(o)$ (Steps 1–3) and the pond P (Step 4).

Step 0: From contour lines to a grid (contour sources only)

A contour map gives elevations only *on* the lines. The planner has to guess the ground between them, and the guess matters for storage. Figure 12 shows a synthetic ridge drawn as 20 m contours and gridded both ways by `_grid_from_contours`. The linear surface stops at the top contour: every triangle up there has three vertices on the same line. It flattens the summit at 380.0 m, while the true top is 394.9 m. Inside the closed 340 m contour, 17.6 % of the linear cells are exactly flat and 0.0 % of the bounded smooth cells are. A pond placed on a flat terrace would have a storage of footprint $\times$ stage, which is too much.

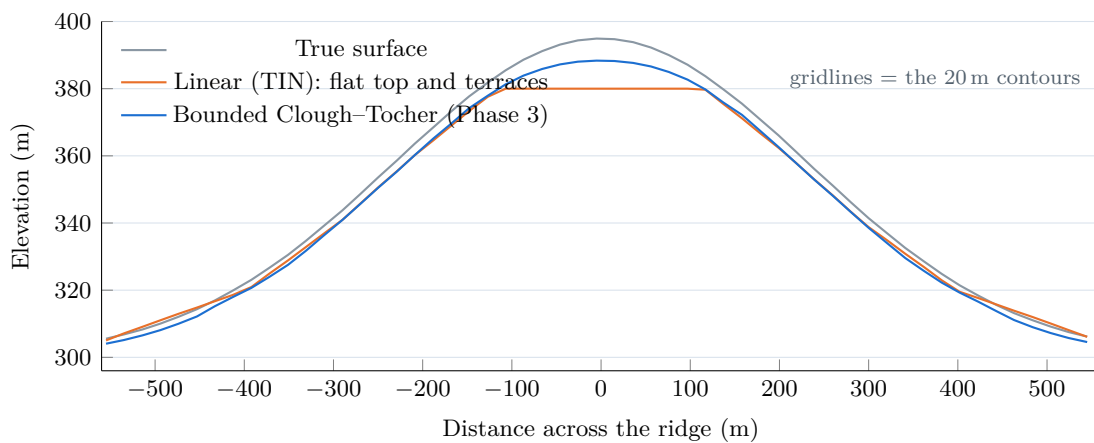


Figure 12: Profile through a synthetic ridge after gridding its 20 m contours. The bounded smooth surface follows the true shape between contours and never crosses the next contour level.

Step 1: The elevation grid and the pit

Figure 13a is the grid the analysis starts from. Rows run north (0) to south (6) and columns west (0) to east (6); darker cells are higher. Cell (2,3) is a *pit*: at 100.9 m it is lower than all eight of its neighbours, so water routed into it would have nowhere to go.

Step 2: Fill the pit (priority flood)

Routing needs every cell to have a way out. The priority flood (Equation 3) starts from all edge cells in a queue ordered by elevation. It always takes the lowest cell out of the queue and raises each unvisited neighbour to at least that cell's height plus ε . The water level therefore rises from the outside in, like a tank filling. The pit is reached from its lowest neighbour, (3,3) at 101.8 m, and is raised to $101.8 \text{ m} + \varepsilon$, just above that neighbour (Figure 13b). It now drains south into (3,3). No other cell changes.

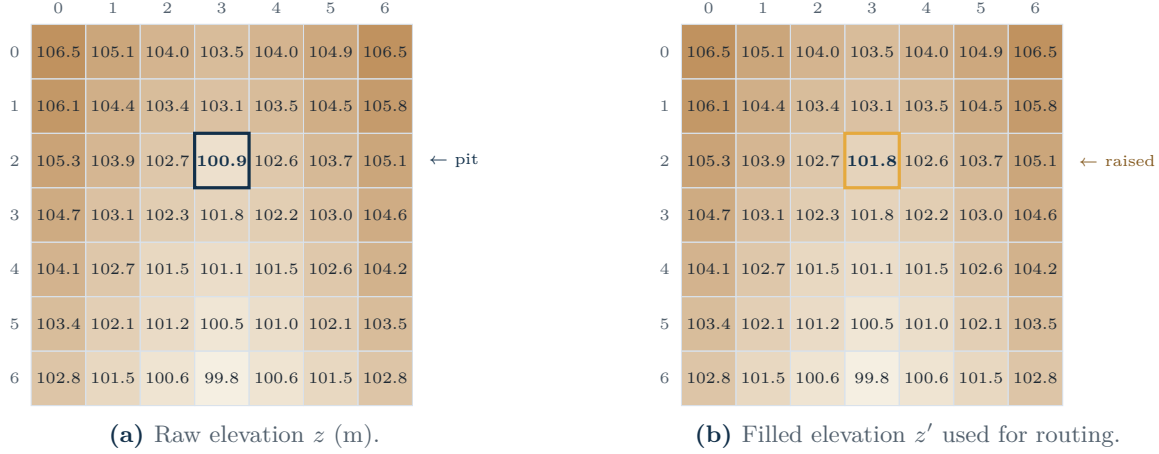


Figure 13: The worked-example grid (20 m cells, 99.8–106.5 m). Only the pit changes when the grid is filled. Storage is later measured on the *raw* grid, so the pit still holds water in the pond.

Step 3: Flow directions, accumulation and the catchment

Each cell drains to the neighbour with the steepest downhill grade (Equation 4). Table 3 does this by hand for cell (5,2) (101.2 m), just west of the outlet. A diagonal step is $\sqrt{2} \times 20 = 28.3 \text{ m}$ long, so a drop of 1.4 m over 28.3 m (0.0495) beats 0.7 m over 20 m (0.0350). The cell drains south-east to (6,3), past the outlet, not into it.

Neighbour	z'_n (m)	d (m)	Drop $z'_i - z'_n$ (m)	Grade
(4,1)	102.7	28.3	-1.5	-0.0530
(4,2)	101.5	20.0	-0.3	-0.0150
(4,3)	101.1	28.3	+0.1	+0.0035
(5,1)	102.1	20.0	-0.9	-0.0450
(5,3)	100.5	20.0	+0.7	+0.0350
(6,1)	101.5	28.3	-0.3	-0.0106
(6,2)	100.6	20.0	+0.6	+0.0300
(6,3)	99.8	28.3	+1.4	+0.0495

Table 3: The D8 choice for cell (5,2). Negative grades are uphill and never chosen; the steepest downhill neighbour (bold) wins.

Following the arrows downstream and counting how many cells pass through each cell gives the accumulation $A(i)/A_{\text{cell}}$ (Equation 5); the orange numbers in Figure 14a. The catchment of the outlet $o = (5, 3)$ is every cell whose arrows lead to it: 33 cells, so

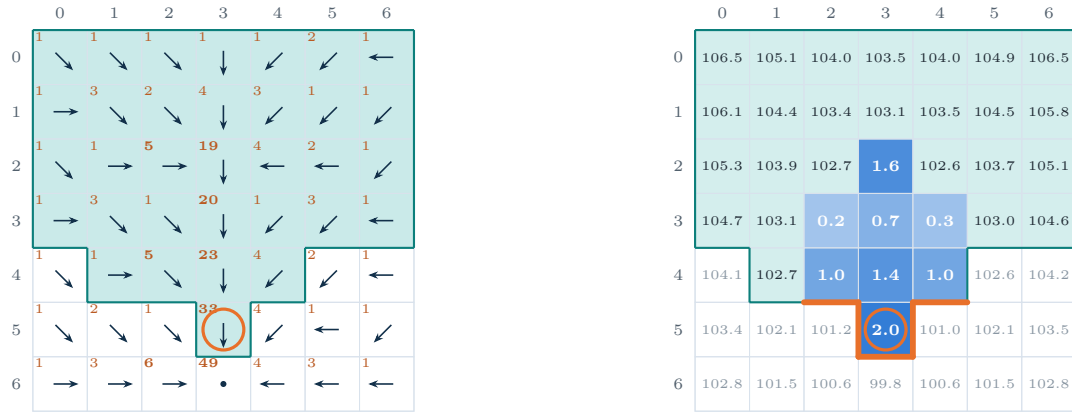
$$A_C = 33 \times 400 \text{ m}^2 = 13,200 \text{ m}^2 = 1.32 \text{ ha}.$$

The cells either side of the outlet are *not* in it, as Table 3 showed. This will decide where the bund goes.

Step 4: Flood the pond to the crest

Put a 2 m embankment at the outlet, whose ground is $z_o = 100.5 \text{ m}$. Water can rise to the crest

$$h_c = z_o + h = 100.5 + 2 = 102.5 \text{ m}.$$



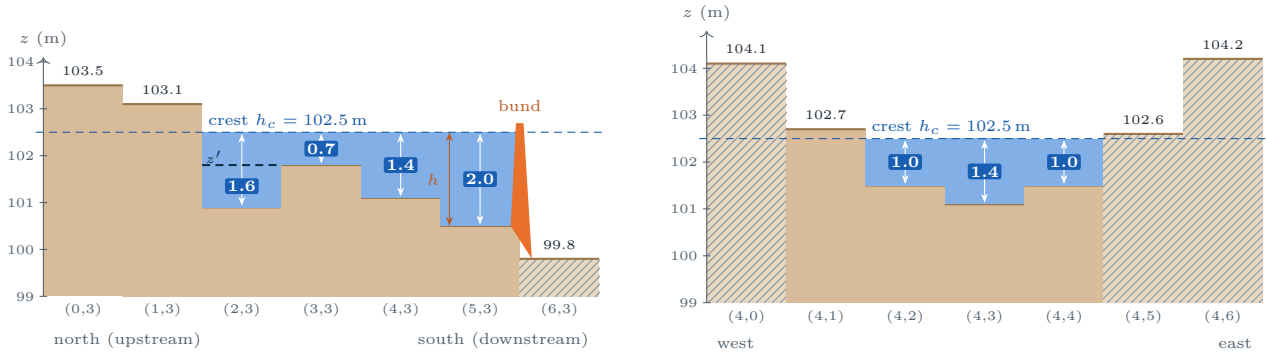
(a) D8 arrows, accumulation in cells (orange), and the catchment $C(o)$ of the outlet (teal, ringed). (b) Pond at $h = 2$ m: water depth $h_c - z_j$ (m) in blue cells, bund edges — and ground elevations elsewhere.

Figure 14: Plan views of the worked example. ■ catchment $C(o)$ ■ pond P — embankment edge ○ outlet o .

Starting at the outlet, the flood fill steps to 4-neighbours that satisfy Equation 6. Each such cell lies in the catchment, is no higher than the crest, is allowed land, and is within 100 m. 8 cells qualify (Figure 14b). The pit joins at this stage because its neighbour (3,3) floods first. Each cell holds a column of water $h_c - z_j$ deep, so

$$\begin{aligned}
 V_S &= \sum_{j \in P} (h_c - z_j) A_{\text{cell}} = (2.0 + 1.0 + 1.4 + 1.0 + 0.2 + 0.7 + 0.3 + 1.6) \times 400 \\
 &= 8.2 \text{ m} \times 400 \text{ m}^2 = \mathbf{3,280 \text{ m}^3}, \\
 A_P &= 8 \times 400 = 3,200 \text{ m}^2, \quad \bar{d} = V_S / A_P = 1.03 \text{ m}.
 \end{aligned}$$

Water escapes wherever a pond cell touches lower ground *outside* the catchment. There are 5 such cell edges: the outlet's own south, west and east edges, and the south edges of the two cells beside it. Those are the cells from Table 3 that drain past the outlet. So the indicative bund is $L_E = 5 \times 20 = 100$ m. Figure 15 shows the same pond in cross-section, and Figure 16 in 3D.



(a) Along the valley floor (column 3). The dashed navy line is the filled pit; the bund closes the outlet's downstream edge. (b) Across the valley (row 4). Hatched cells lie outside $C(o)$ and never count as storage, even where they are low.

Figure 15: Cross-sections of the worked pond. Each blue column is one cell's water depth $h_c - z_j$; the storage is their sum times the cell area. Vertical scale exaggerated.

Step 5: The stage-storage curve

Repeating Step 4 for other embankment heights gives Table 4. Storage grows faster than stage, because a higher crest both deepens the flooded cells and floods new ones. The area column jumps in whole cells, and a longer bund is needed each time the water reaches another low edge outside the catchment. At 0.5 m only the outlet cell floods ($0.5 \times 400 = 200 \text{ m}^3$); the pit joins at 2 m. This is the curve the planner draws for every site.

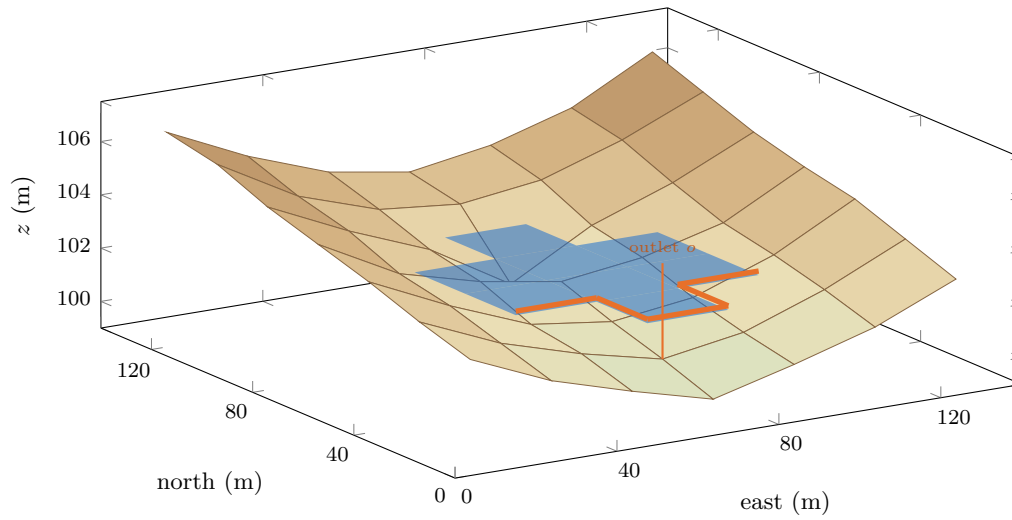


Figure 16: Wireframe of the worked grid (cell centres, vertical scale exaggerated) with the pond surface at the crest (102.5 m, blue) and the bund edges (orange). The planner’s 3D inspector (Figure 9) draws the real grid the same way.

h (m)	A_P (m ²)	V_S (m ³)	L_E (m)
0.5	400	200	40
1.0	1,600	560	100
1.5	2,400	1,880	100
2.0	3,200	3,280	100
2.5	4,800	5,280	160
3.0	6,800	8,040	160

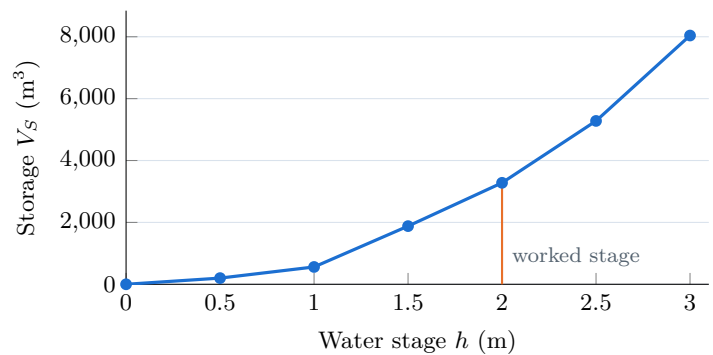


Table 4: Stage–area–storage–bund table of the worked pond, computed by `stage_curve`, and its curve.

Step 6: Runoff and the collectable volume

Worked calculation: water balance of the example pond

Rainfall $P_{\text{mm}} = 1,287 \text{ mm}$ (the June–September 2025 total used for Kanker west below), runoff coefficient $C = 0.35$ (hilly land), catchment $A_C = 13,200 \text{ m}^2$:

$$V_R = \frac{P_{\text{mm}}}{1000} C A_C = \frac{1,287}{1000} \times 0.35 \times 13,200 = 5,946 \text{ m}^3.$$

$$V_c = \min(V_R, V_S) = \min(5,946, 3,280) = \mathbf{3,280 \text{ m}^3} \quad (\text{limited by storage}), \quad F = V_R/V_S = 1.81.$$

The season delivers 1.81 times what the pond holds. The pond fills, the surplus must pass a safe overflow, and the planner shows the “limited by pond size” label and a spillway warning when $F > 3$.

6.1 The same calculation on a real site: Kanker west

The best site in the Kanker west example (Copernicus GLO-30, $\approx 30 \text{ m}$ cells) runs through exactly the same steps on a 280×280 grid. Its values come from the planner’s API response, and Figure 6 shows the same values in the interface:

Worked calculation: Kanker west, site 1 at $h = 2.5 \text{ m}$

Rainfall, CHIRPS Jun–Sep 2025	$P_{\text{mm}} = 1,287.3 \text{ mm}$
Catchment $C(o)$	$A_C = 31.07 \text{ ha} = 310,730 \text{ m}^2$
Runoff	$V_R = \frac{1287.3}{1000} \times 0.35 \times 310,730 = 140,002 \text{ m}^3$
Pond at crest $h_c = 423.4 \text{ m}$	$A_P = 22,517 \text{ m}^2$, $\bar{d} = 2.38 \text{ m}$, $V_S = 53,625 \text{ m}^3$
Bund	$L_E \approx 180 \text{ m}$
Collectable	$V_c = \min(140,002, 53,625) = \mathbf{53,625 \text{ m}^3}$ (storage-limited, $F = 2.61$)

That is more than the Mission Amrit Sarovar plains benchmark (at least 1 acre and about $10,000 \text{ m}^3$). It equals about 61 MGNREGA model farm ponds of 880 m^3 . The stage–storage curve (Figure 17) shows how a higher bund trades embankment length for water: the season’s runoff would fill the pond to about 6 m.

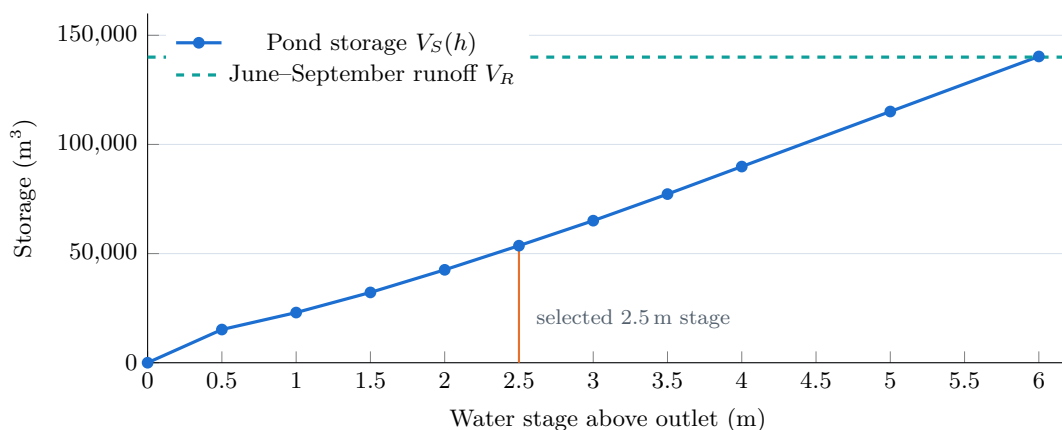


Figure 17: Stage–storage curve for the best Kanker west site, from the API response. Storage rises with stage while the embankment grows from 30 m to 240 m.

6.2 Reading the result on the map

Figure 18 labels each element of a finished analysis on the real map, using the supplied Khapri contours.

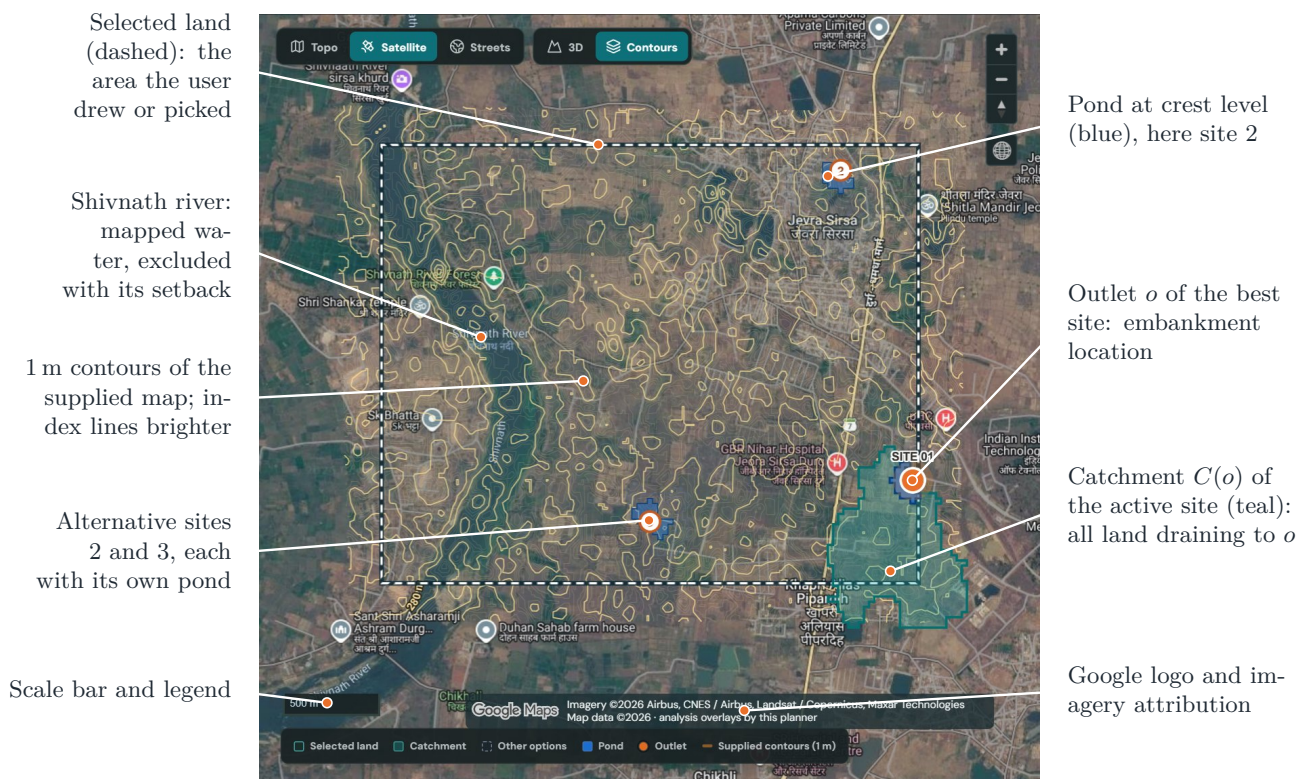


Figure 18: Annotated map of the Khapri analysis (crop of Figure 2). Site 1 drains 41.9 ha and stores 47,291 m³ at 2.5 m; its catchment runs beyond the supplied contours, so the planner warns that its area may be underestimated.

7 Data sources, provenance and licences

Source	Used for	Licence and attribution
Copernicus DEM GLO-30 via AWS Open Data [2] (fallback: Planetary Computer)	Analysis elevation (surface model, EGM2008 heights), contour export	© DLR e.V. 2010–2014 and © Airbus Defence and Space GmbH 2014–2018, provided under COPERNICUS by the EU and ESA
CHIRPS v3 monthly [3]	Rainfall	CC BY 4.0; doi:10.15780/G2JQ0P
OpenStreetMap via Overpass [4]	Water and built-land screening	ODbL, © OpenStreetMap contributors
Google Map Tiles API [6] (optional)	Satellite basemap and 3D drape	Google Maps Platform terms: display only, no caching or storage, Google logo and viewport attribution shown
Google Maps JavaScript API, Photorealistic 3D (optional)	Google 3D Earth view	As above; attribution drawn by Google’s element
Google Places API (New) (optional)	Place search	As above; “Google Maps” logo on the results
OpenTopoMap	Topographic basemap	CC-BY-SA
EOxCloudless 2025 [5]	Satellite basemap and drape without a Google key	CC BY-NC-SA 4.0: non-commercial academic use
OpenStreetMap tiles	Street basemap	OSMF tile usage policy
Terrain Tiles on AWS	MapLibre 3D terrain only	SRTM and GMTED2010 courtesy of USGS; ETOPO1 courtesy of NOAA
Photon (komoot)	Place search without Google	OSM data; public server used within fair use

Two sources were deliberately rejected after checking their terms:

- **Esri World Imagery** requires an ArcGIS account for this kind of use.
- **The OpenStreetMap editing API**, which the prototype used, is reserved for editing, not read-only applications. It also refused a 14×12 km area.

The Google key is a browser key: any page that uses it exposes it to the browser. It is therefore kept out of the public repository (in a git-ignored `.env` on the server), and its quota is limited on Google’s side. Screenshots with Google content keep the logo and attribution, as Google’s guidelines for maps in documents require [7].

Provenance of the supplied contour map

`contours_1m.kml` covers 81.281–81.313°E, 21.240–21.264°N. That is Khapri village farmland between the IIT Bhilai campus (centre about 81.318°E, 21.245°N) and the Shivnath river, and only a thin strip overlaps the campus. Its internal structure (a root `Folder` named `ContourMapGenerator` and lxml `py:pytype` attributes) matches exports from the free Contour Map Generator [15], which traces roughly 30 m satellite elevation. The 1 m contour interval is therefore finer than the real precision of the data. The planner labels it as supplied contours, not a field survey, and says so in its limitations.

Better sources for detailed design include:

- **Survey of India Open Series Maps:** free 1:50,000 GeoPDFs with 20 m contours, which need digitising [16];
- **ISRO Bhuvan CartoDEM v3:** 30 m, login required;
- **FABDEM:** bare-earth 30 m, non-commercial;
- **a village DGPS or drone survey** exported as contour KML.

Any DEM can be converted with `gdal_contour -f KML -i 5 -a elevation dem.tif out.kml`, and the planner reads the `elevation` attribute.

8 Verified example areas

Each area was run end to end with the final algorithm: GLO-30 elevation, the CHIRPS June–September 2025 total, Overpass screening, three ranked sites and contour export. Its contours are shipped in `contour-maps/real/`. The areas were chosen for real hill-to-plain relief with villages and seasonal drainage. The Ralegan Siddhi, Hiware Bazar, Alwar, Sukhomajri and Abreha we Atsbeha areas are known for community watershed work.

Area	Region	Relief (m)	Monsoon (mm)	Site 1 (ha)	Site 1 (m ³)
Khapri (supplied contours)	Durg, beside IIT Bhilai	266–298	1,309	41.9	47,291
Dongargarh–Khairagarh ridge	~48 km W	338–632	1,337	34.0	44,051
Maikal ridge front	Kabirdham, ~90 km N	353–741	1,141	27.2	41,544
Kanker west	~110 km S	392–682	1,287	31.1	53,625
Sihawa, Mahanadi source	Dhamtari, ~120 km SE	426–550	1,287	56.4	43,319
Mainpat plateau	Surguja, ~265 km NE	1,026–1,120	1,626	21.3	15,196
Ralegan Siddhi	Maharashtra	611–747	623	25.8	24,315
Hiware Bazar	Maharashtra	684–876	665	97.0	32,525
Bheekampura johads	Alwar, Rajasthan	393–634	1,130	17.2	39,658
Sukhomajri	Panchkula, Haryana	426–606	1,538	26.4	23,998
Abreha we Atsbeha	Tigray, Ethiopia	1,926–2,539	681	54.2	26,191

Table 5: Example areas (June–September 2025 CHIRPS, $C = 0.35$, 2.5 m stage). Site 1 is the best-ranked outlet; in every area it is limited by storage, not runoff. The Khapri row uses the supplied contours, so its relief is the grid’s, not GLO-30’s.

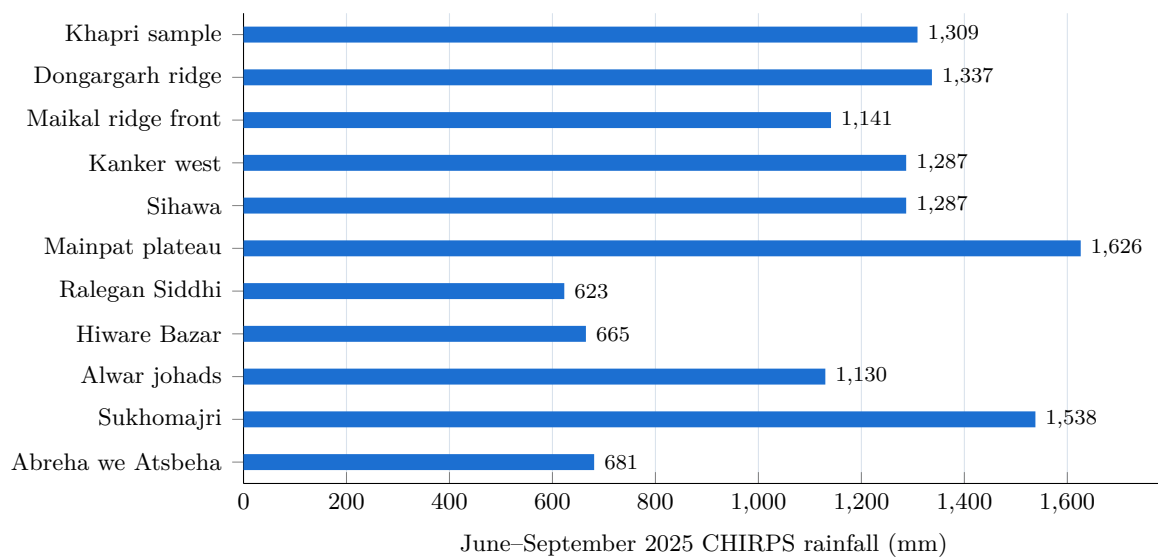
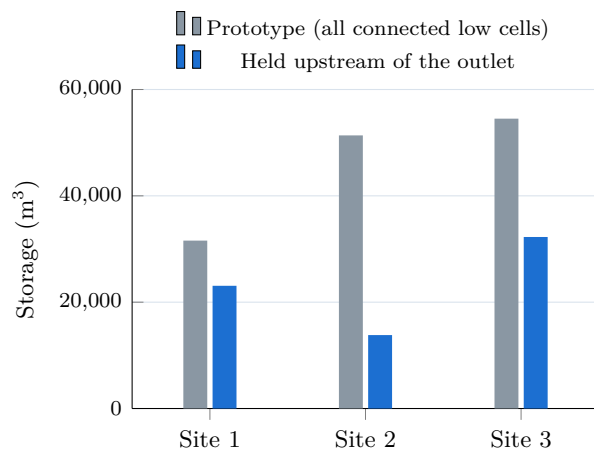


Figure 19: Monsoon rainfall read from CHIRPS for each example. The drought-prone Maharashtra villages receive half the rainfall of the Chhattisgarh sites. For comparison, the IMD 1991–2020 normal for Raipur is about 1,145 mm for June–September.

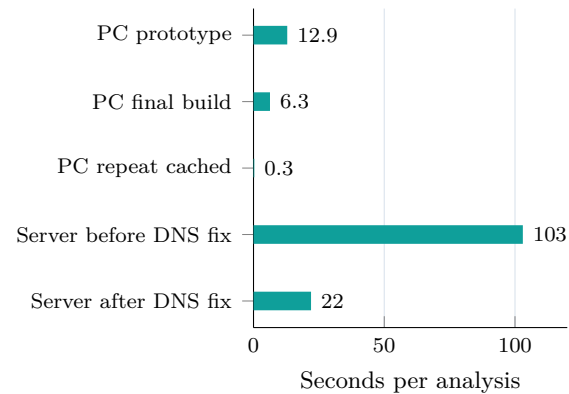
9 Quality review: defects found and fixed

The Phase 3 prototype (26 September 2026) was reviewed with probe scripts, live network tests and browser screenshots. Every defect below was reproduced before it was fixed and is now covered by a test.

Area	Defect (measured)	Fix
Pond storage	Pond cells spread downstream and sideways outside the outlet's own catchment. That was 25–54 % of each footprint, and it inflated storage by 37–272 % (Figure 20a).	Impoundment model restricted to $C(o)$; the embankment length comes from lower ground outside it (Section 6, Step 4).
Ranking	Sites were the cells just below an arbitrary 98th-percentile “channel” cutoff. All three had 32–36 ha catchments on the sample (cutoff 36.3 ha), and storage never affected the rank.	40 separated candidates ranked by collectable volume; an explicit small-pond catchment cap.
Terraces	Linear interpolation of 20 m contours left 22.5 % of cells flat. Two sites shared one drainage line, and their storage was footprint $\times$ stage.	Bounded Clough–Tocher surface (0.6 % flat, Figure 12) and catchment-diversity rule.
OSM screening	The prototype used the editing API against its policy, and a 14 $\times$ 12 km area returned “unavailable”. It applied a 35 m buffer to every footpath.	Overpass with three servers, partial answers rejected, buffers by road class. The same area now screens 249 water and 15,361 built features.
Rainfall	A single month (August 2025) was treated as the planning rainfall.	Monsoon and annual CHIRPS totals fetched in parallel, with monthly bars.
Speed	DEM, OSM and CHIRPS were fetched one after another: 12.9 s for a Copernicus area.	Parallel fetches, a grid cache, AWS tiles: 6.3 s cold, 0.3 s repeat (Figure 20b).
Map	Switching the terrain source destroyed the map, so the drawn area vanished while the panel still said “Area selected”.	The map is created once and its layers are updated in place.
3D view	A fixed 15 $\times$ exaggeration made plains look like spikes, and the skirt dominated the view.	Automatic exaggeration, a thin skirt, draped contours, satellite drape.
Usability	The only way to reach another area was panning; drawing outside the survey just produced an error.	Search, examples, automatic source switching, a live area check.
Accessibility	About 30 text colours were below WCAG AA (e.g. #99a5a9 on white $\approx$ 2.6:1), and some text was 8–9 px.	Colours darkened to at least 4.5:1; minimum sizes raised.
Honesty	The sample was described as a campus survey, and the runoff presets had no source.	Provenance corrected; presets cite CGWB ranges.



(a) Storage inflation in the prototype (sample area).



(b) Analysis time for a Copernicus area. Server values are the worst case of the runs (69–103 s before, 3–22 s after).

Figure 20: Two of the measured improvements.

Defects found in the new Google and theme work before release

Feature	Defect (how it was found)	Fix
Google 3D Earth	Google's <code>Polygon3DElement</code> throws on an empty <code>innerCoordinates</code> list, and the uncaught error blanked the whole planner (browser test).	Holes are passed only when present; each overlay is added in its own guard, and an error boundary isolates every optional Google feature.
Google Places	<code>crypto.randomUUID</code> exists only on HTTPS or localhost. On the plain-HTTP IIT address the fallback token made every search return HTTP 400 (live test).	Session tokens built from <code>crypto.getRandomValues</code> .
Dark theme	The generator only read hex colours, so rules written as <code>white</code> stayed light (the 3D inspector header), and it brightened pale chart gridlines into bright lines (screenshot review).	Named colours are mapped too; pale strokes are treated as borders; chart series have validated dark steps.
Browser tests	With satellite as the default basemap, tiles cancelled by zooming (<code>ERR_ABORTED</code>) were counted as failures.	Cancelled requests are ignored; real failures still fail, and logged URLs drop their query strings, so the key is never printed.

10 Validation

10.1 Automated tests

Suite	Tests	What is checked
<code>test_api.py</code>	6	Phase 2 route: health, sample KML, KMZ, unsupported format; <code>/api/config</code> exposes a Google key only when one is configured; the <code>.env</code> loader never overrides real environment variables
<code>test_planning.py</code>	6	Geometries and the runoff formula; different land gives different sites; outside-area rejection; contour overlay; sites and ponds avoid mapped water and roads; OSM XML parsing
<code>test_hydrology.py</code>	11	Pond inside its own catchment; ranking by collectable volume; consistent stage curves; non-overlapping ponds; higher stage holds more; contours in the response; a synthetic valley; rainfall periods; Overpass parser and road classes; contour export round trip
<code>test_upload.py</code>	8	Uploaded KML/KMZ drives the analysis; absolute altitudes and ExtendedData; every shipped real KML uploads and analyzes; bounded smooth interpolation; distinct drainage lines
<code>test_resilience.py</code>	5	AWS tile naming across hemispheres and seams; retries; disk cache; offline cached DEM; DNS stale-if-error

All 36 tests pass on the development PC and on the IIT server. Five Playwright suites drive a real Chrome, on desktop (1440×900) and mobile (390×844):

- `smoke.mjs`: analyze, draw a second area and confirm the site changes;
- `smoke-upload-3d.mjs`: upload two contour maps, open the 3D model, orbit it, and change site and exaggeration;
- `walkthrough.mjs`: satellite and 3D terrain, source switching, an example area, the inspector with satellite drape, the globe, and search;
- `google-check.mjs`: the Google tile session and viewport attribution, Places search and lookup, the Google drape with its credit, and the 3D Earth view with all ten overlays;
- `theme-review.mjs`: every main screen in both themes, including the phone layout and the Earth view, for visual review.

Setting `POND_URL=http://10.1.75.53:3233/` runs them against the live deployment, where they pass with no page errors. The worked example in Section 6 is itself a check: `report_figures.py` asserts that its hand-derived bund edges give the same length as `impoundment()`.

10.2 Live verification

Request on the deployed server (from the campus network)	HTTP	Time
GET /, /docs, /health, /api/config, front-end assets (JavaScript served as text/javascript)	200	<0.1 s
Sample analysis with the CHIRPS monsoon total	200	1.2 s
Cached example areas (Maikal ridge front, Abreha we Atsbeha)	200	0.7–1.1 s
KML upload (106 contour lines) / contour KML export	200	0.1 s
Phase 2 POST /analyzeContour with the supplied KML	200	2.9 s
New, uncached areas (AWS DEM, CHIRPS and Overpass fetched live)	200	3–22 s
Cached example with <i>all</i> network access blocked (proxy test)	200	0.4 s
Google layers from the live page (google-check.mjs)	pass	no problems

11 Deployment on the IIT system

The application runs on the assigned container: SSH port 2233, so Application 1 is internal port 3000, published as external port 3233. It runs in tmux session **pond-app** under a loop script (**scripts/run_server.sh**) that restarts Uvicorn if it exits and logs to **logs/app.log**. The optional Google key is read from **~/ai-village-pond-planning/.env** (permissions 600, not in git). Front-end updates are deployed by replacing the built **frontend/dist** folder; the server needs no restart for them. Figure 21 shows the API documentation.

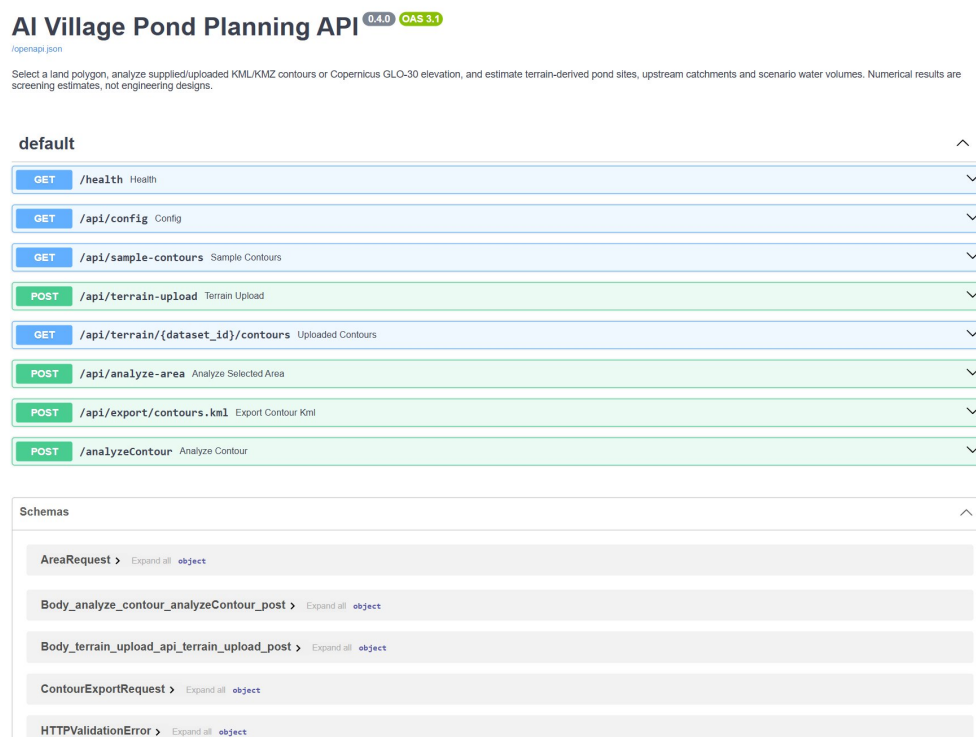


Figure 21: Swagger UI of the API (served at <http://10.1.75.53:3233/docs>), listing the eight routes.

11.1 Constraints found and how they were handled

Constraint (measured)	Handling
No Node.js; PyPI downloads failed DNS or TLS checks	The front end is built on a workstation. Linux wheels (73 packages, 101 MB) are installed offline with <code>pip --no-index</code> . <code>uvloop</code> is added explicitly, because pip on Windows skips Linux-only dependencies.
SFTP restricted	Files are streamed through the SSH channel and verified by SHA-256.
Windows line endings broke the start script	<code>.gitattributes</code> keeps <code>*.sh</code> as LF.
Container DNS (1.1.1.1/8.8.8.8) drops lookups at random, about 20 s per failure	Retries on every remote read; an in-process DNS cache that reuses the last good address when a lookup fails; data hosts resolved at start-up; AWS as the primary DEM host. A new area went from 69–103 s with screening unavailable to 3–22 s fully screened.
Demo reliability	The 11 example areas are pre-cached (DEM, CHIRPS, OSM), so they need no outbound access.
From campus Wi-Fi (10.50.x), about 50 % of new TCP connections to 10.1.75.53 fail (10/20 on port 3233, 11/20 on port 2233), while 20/20 succeed from inside the server network	This loss is outside the application. The front end retries API calls after network-level failures, and the test scripts retry the first page load. A dropped stylesheet or tile request can still leave a page half-drawn; refreshing fixes it, and wired LAN is recommended for the demonstration.
The container restarted once during the work and took the tmux session with it; it has no init system (PID 1 is <code>sshd</code>)	The session was restarted by hand with the command in Appendix B; checking <code>/health</code> before a demonstration is part of the demo script.

12 Limitations and responsible use

- **Elevation.** GLO-30 is a 30 m *surface* model, so trees and buildings raise it and it misses small channels and field bunds. Contour grids are interpolations. A catchment that reaches the grid edge is flagged as possibly underestimated.
- **Hydrology.** The runoff coefficient is a seasonal fraction, and CHIRPS pixels (≈ 5 km) are wider than a village. Collectable volume is one fill, not an annual yield. Seepage, evaporation (roughly 1.3–1.6 m a year near Raipur), siltation and spillway design are not modelled.
- **Screening.** OpenStreetMap is incomplete in rural Chhattisgarh. Forest land, sanctuaries and protected areas are not screened. Land ownership and legal eligibility are unverified; every site is labelled *unverified*.
- **Scale.** The pond radius cap keeps results at village-tank scale, so larger reservoirs are underestimated by design.
- **Google layers.** Google's imagery and 3D surface are for visual context and may differ in date and alignment from the elevation grid. They depend on the key's daily quota; when it is exhausted the planner falls back to the open layers.
- **Imagery licence.** EOxCloudless imagery is licensed for non-commercial use only.

Every candidate needs a field visit, a topographic survey, soil tests and land records before any construction decision.

13 Disclosure of AI use

Generative AI was used extensively in the final Phase 3 build, for coding, debugging and documentation. This section states what it was used for, what it was not used for, and how its output was checked.

Tool	Claude Code (Anthropic), running the Claude Opus 5.5 model, used on 27–28 September 2026 on the development PC [18]. It ran research sub-agents that used Firecrawl web search and scraping [19].
Coding	Wrote most of the final Phase 3 code and tests: • backend: the impoundment and ranking model, bounded smooth interpolation, the Overpass screening, rainfall periods, the retry, cache and DNS layers, contour tracing and export, and the <code>.env</code> key loading; • front end: search, examples, basemaps, 3D terrain and globe, the results panel and charts, the 3D inspector changes, exports and the contrast fixes; • the optional Google layers (tiles with attribution, 3D Earth view, Places); • the dark-theme generator and its chart palette; • this report’s figure generator. Every commit from <code>af86e61</code> onwards carries a Co-Authored-By: Claude trailer.
Debugging	Reviewed the prototype for defects and wrote probe scripts that measured them (storage inflation, ranking clustering, failed OSM requests, the map rebuild, flat terraces). Found the Google-layer and theme defects in Section 9 through browser tests and screenshot review. Diagnosed deployment problems: restricted SFTP, CRLF endings, shell path conversion, missing Linux wheels, container DNS, campus packet loss and the container restart.
Research	Found and checked data sources, tile and API terms (including Google’s attribution and no-caching rules), the IIT Bhilai campus coordinates, the provenance of the supplied KML, example areas, and planning norms (Amrit Sarovar, MGNREGA, CGWB). Research claims were cross-checked by running the pipeline on each area and by reading the primary documents cited in the references.
Documentation	Drafted the README, HANDOVER.md, code comments and this report, including its tables, equations and diagrams. The worked-example figures are drawn by a script from values the planner’s code computes; none were drawn by hand or by an image model.
Operations	Ran the tests, builds, browser suites, screenshot capture and deployment commands over SSH. It used credentials and the Google key supplied by the team. It asked for approval before destructive or public actions (cleaning the server, pushing to GitHub), and kept the key out of every commit.

- **Not used for results.** The planner makes no calls to a language model. Every number in the application and in this report comes from deterministic code run on the cited data, or from measured test runs.
- **No generated images.** Every figure is either a screenshot of the running system or a chart or diagram drawn from computed or measured values. The conceptual images in `output/imagegen/` come from earlier phases and are not used here as evidence.
- **Human responsibility.** The team set the goals and constraints and made the decisions: what to fix, what to clean up, which Google features to use, when to deploy and publish. It provided access and reviewed the results, and remains responsible for the submission.

- **AI errors that were caught.** Checking the AI's output changed it in several places:
 - the sample was first described as the IIT Bhilai campus, and was corrected after the location was verified;
 - the first runoff presets had no source and were replaced by the CGWB ranges;
 - a first cubic interpolation overshot the survey by 20 m and was bounded;
 - a flat-terrace artefact was found by inspecting a live screenshot and was fixed;
 - the first Google 3D Earth version crashed the whole page, and Places search failed on the HTTP address (Section 9);
 - the AI once reported temporary probe files as deleted before they were; this was noticed and corrected;
 - in this report, a first draft of the Kanker storage read $53,624 \text{ m}^3$ (half-to-even rounding of 53,624.5) against the planner's 53,625; the figure script now rounds halves up, as the interface does.

14 Future work

- A verified government-land parcel layer (e.g. Bhulekh or DILRMP data) as an eligibility mask, and forest and sanctuary boundaries.
- A bare-earth DEM (FABDEM or CartoDEM), and on-site drone or DGPS surveys, for design-grade storage.
- Monthly water-balance simulation with evaporation, seepage and demand, with a spillway check for large catchments.
- Land-cover-weighted runoff from ESA WorldCover instead of one coefficient.
- Restricting the Google key to the planner's address in the Google Cloud console, and a service manager on the container so the planner restarts by itself.
- Durable, per-user storage of uploads (currently in memory for two hours) and a stable public mirror.

References and links

- [1] Project repository: <https://github.com/Galabavamsi/ai-village-pond-planning>
- [2] Copernicus DEM on AWS Open Data: <https://registry.opendata.aws/copernicus-dem/>; Planetary Computer: <https://planetarycomputer.microsoft.com/dataset/cop-dem-glo-30>
- [3] CHIRPS v3: <https://www.chc.ucsb.edu/data/chirps3>, doi:10.15780/G2JQ0P
- [4] Overpass API: https://wiki.openstreetmap.org/wiki/Overpass_API; OSM licence: <https://www.openstreetmap.org/copyright>
- [5] EOxCloudless: <https://cloudless.eox.at>; OpenTopoMap: <https://opentopomap.org/about>; AWS Terrain Tiles: <https://registry.opendata.aws/terrain-tiles/>; Photon: <https://photon.komoot.io>
- [6] Google Map Tiles API policies: <https://developers.google.com/maps/documentation/tile/policies>; Photorealistic 3D Maps in the Maps JavaScript API: <https://developers.google.com/maps/documentation/javascript/3d-maps-overview>; Places API (New): <https://developers.google.com/maps/documentation/places/web-service/op-overview>
- [7] Google Maps, Google Earth and Street View guidelines (use in documents and screenshots): <https://www.google.com/permissions/geoguidelines/>

-
- [8] Mission Amrit Sarovar, Phase-2 guidelines (MoRD, 2024): <https://amritsarovar.gov.in/AmrutSarovarDocuments/AmritSarovarGuidelinesPhase2.pdf>
 - [9] MGNREGA suggestive estimates for farm ponds (MoRD circular, 2016): <https://nregaplus.nic.in/netnrega/writereaddata/Circulars/1569SuggestiveEstimatesFarmPondNADEPandVermiCompost.pdf>
 - [10] CGWB, Manual on Artificial Recharge of Ground Water: <https://cgwb.gov.in/cgwbpm/public/uploads/documents/1679997242339591060file.pdf>
 - [11] R. Barnes, C. Lehman, D. Mulla, “Priority-flood: An optimal depression-filling and watershed-labeling algorithm for digital elevation models”, *Computers & Geosciences* 62 (2014) 117–127.
 - [12] J. F. O’Callaghan, D. M. Mark, “The extraction of drainage networks from digital elevation data”, *Computer Vision, Graphics, and Image Processing* 28 (1984) 323–344.
 - [13] SciPy CloughTocher2DInterpolator and griddata: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.CloughTocher2DInterpolator.html>
 - [14] B. Ottosson, “A perceptual color space for image processing” (OKLab): <https://bottosson.github.io/posts/oklab/>
 - [15] Contour Map Generator: <https://map.contourmapgenerator.com>
 - [16] Survey of India Online Maps Portal: <https://onlinemaps.surveyofindia.gov.in>; Bhuvan: <https://bhuvan-app3.nrsc.gov.in/data/download/index.php>; FABDEM: <https://data.bris.ac.uk/data/dataset/s5hqmjcdj8yo2ibzi9b4ew3sn>; OpenTopography API: <https://portal.opentopography.org/apidocs/>
 - [17] MapLibre GL JS: <https://maplibre.org>; Terra Draw: <https://github.com/JamesLMilner/terra-draw>; Three.js: <https://threejs.org>; FastAPI: <https://fastapi.tiangolo.com>; contourpy: <https://contourpy.readthedocs.io>; rasterio: <https://rasterio.readthedocs.io>; Lucide icons: <https://lucide.dev>
 - [18] Claude Code: <https://claude.com/claude-code>
 - [19] Firecrawl: <https://www.firecrawl.dev>

A API reference

Route	Purpose
POST /api/analyze-area	JSON: area (GeoJSON Polygon), source (sample upload copernicus), dataset_id, rainfall_source (chirps manual), rainfall_period (monsoon annual month), rainfall_year, rainfall_month, rainfall_mm, runoff_coefficient, stage_m, max_catchment_ha
POST /api/terrain-upload	Multipart contour_map (KML/KMZ, ≤20 MB) → temporary dataset_id, bounds, elevation range
GET /api/terrain/{id}/contours	Uploaded linework as GeoJSON
POST /api/export/contours.kml	Contour KML for area from the chosen source (optional interval_m)
GET /api/config, /api/sample-contours	Sample extent, defaults, example areas and the optional google_maps_key; supplied contours
POST /analyzeContour, /findCatchment	Phase 2 route (multipart contour_map)
GET /health, /docs	Status; Swagger UI

Errors use standard status codes: 400 for an empty upload, 413 for an upload that is too large, 415 for a wrong file type, and 422 for an invalid area, unusable contours or unavailable elevation. Each response recommendation contains:

- location, pond_region and catchment.geometry as GeoJSON;
- pond: storage, footprint, crest, mean and maximum depth, embankment length and stage_curve;
- water: runoff, collectable volume, fill ratio and limited_by;
- site_screening, notes, and land_status, which is always unverified.

The response also carries rainfall (with monthly_mm), water_screening, parameters, contours, terrain_preview and limitations.

```
curl -X POST http://10.1.75.53:3233/api/analyze-area -H "Content-Type: application/json" -d '{
  "area": {"type": "Polygon", "coordinates":
    [[[81.4178,20.2506],[81.4561,20.2506],[81.4561,20.2867],[81.4178,20.2867],[81.4178,20.2506]]]},
  "source": "copernicus", "rainfall_source": "chirps", "rainfall_period": "monsoon", "rainfall_year":
    2025,
  "runoff_coefficient": 0.35, "stage_m": 2.5 }'
```

B Reproduce and run

```
# Development (Windows PowerShell)
py -3 -m venv .venv; .\.venv\Scripts\python.exe -m pip install -r requirements.txt
cd frontend; npm ci; npm run build; cd ..
.\.venv\Scripts\python.exe -m pytest -q           # 36 tests
.\.venv\Scripts\python.exe run.py                 # http://127.0.0.1:8000/
cd frontend; node scripts\smoke.mjs; node scripts\smoke-upload-3d.mjs; node scripts\walkthrough.mjs
node scripts\google-check.mjs; node scripts\theme-review.mjs
python scripts\prewarm_cache.py                   # fill cache/ for the example areas
python scripts\create_copernicus_contour_demo.py  # regenerate contour-maps/real/*.kml
python scripts\build_dark_theme.py                # regenerate frontend/src/dark.css

# Optional Google layers: a git-ignored .env file in the repository root
GOOGLE_MAPS_API_KEY=<browser key>

# This report
node frontend/scripts/report-shots.mjs; python scripts/report_images.py; python scripts/report_figures.py
pdflatex -interaction=nonstopmode -output-directory tmp/latex-build latex/phase3_report.tex # twice

# IIT container (see README "Remote deployment with tmux")
python3 -m venv --without-pip .venv
.venv/bin/python ../wheelhouse/pip-*.whl/pip install --no-index --find-links ../wheelhouse pip
.venv/bin/pip install --no-index --find-links ../wheelhouse -r requirements.txt
tmux new-session -d -s pond-app "bash ~/ai-village-pond-planning/scripts/run_server.sh"
```

C Demonstration script

A walkthrough of about five minutes, in the order the browser tests follow:

1. Check <http://10.1.75.53:3233/health>, then open the planner. Run the supplied contours and explain the three sites, catchments and warnings (Figure 18).
2. Open *Example areas* and choose *Maikal ridge front*; analyze.
3. Switch to 3D over satellite, then open *Google 3D Earth* and step through the three sites.
4. Compare the sites in the table; read the stage-storage chart and the benchmarks.
5. Open *Inspect 3D model* and turn on the satellite drape.
6. Upload `contour-maps/real/kanker-west_glo30.kml` and analyze; export the Google Earth KML.
7. Toggle the light theme once, then close with the limitations: these are screening estimates that need a field survey.

D Repository map

Path	Contents
app/planning.py, hydrology.py	Area validation, grid sources, ranking; upstream index, impoundment, stage table
app/terrain.py, contours.py	KML/KMZ parsing and interpolation (Phase 2 algorithm); contour tracing and KML export
app/osm.py, rainfall.py, cache.py	Overpass screening; CHIRPS periods; retries, disk cache, DNS cache
app/main.py, settings.py, examples.py, upload_store.py	Routes and models; .env and Google key; curated examples; temporary uploads
frontend/src/	React app: App, MapCanvas, ResultsPanel, StageChart, PlaceSearch, TerrainInspector, GoogleEarthView, google, exporters, api; styles.css, planner.css, generated dark.css, dark-extra.css
contour-maps/, contour-maps/real/	Supplied KML, the original GLO-30 fixture, 10 example contour KMLs
tests/, frontend/scripts/	36 pytest tests; browser suites, the Google check, the theme review and the report screenshot script
scripts/	Contour generation, cache pre-warm, server start script, dark-theme generator, report figure and image scripts
latex/, latex/generated/	This report; its generated worked-example figures and numbers
README.md, HANDOVER.md	Operating, deployment and handover documentation